

表中列出的前 4 个原语由服务器按照顺序执行。SOCKET 原语创建一个新的端点 (end point)，并且在传输实体中为它分配相应的表空间。此调用的参数说明了采用的地址格式、所需的服务类型 (比如可靠的字节流)，以及所用的协议。SOCKET 调用成功则返回一个普通的文件描述符，供后续的调用使用，SOCKET 调用与对文件实施的 OPEN 调用工作方式一样。

新近创建的套接字没有网络地址。通过 BIND 原语可以为套接字分配地址。一旦服务器已经将一个地址绑定到一个套接字，则远程客户就能够与它建立连接。之所以不让 SOCKET 调用直接创建一个地址是因为有些进程对于它们的地址比较在意 (比如，它们多年来一直使用同样的地址，所以每个人都知道它们的地址)，而其他的进程并不在乎。

接下来是 LISTEN 调用，它为入境呼叫分配队列空间，以便在多个客户同时发起连接请求时，将这些入境的连接请求排入队列依次处理。与我们第一个例子中的 LISTEN 不同的是，套接字模型中的 LISTEN 并不是一个阻塞调用。

为了阻塞自己以便等待入境连接的到来，服务器执行 ACCEPT 原语。当一个请求连接的段到达时，传输实体创建一个新的套接字并返回一个与其关联的文件描述符，这个新套接字与原来的套接字具有同样的属性；然后服务器可以派生一个进程或者线程来处理这个新套接字上的连接，而服务器自身又回到原来的套接字上等待下一个连接请求的到来。ACCEPT 返回一个文件描述符，服务器可以按照标准的方式对它进行读或者写操作，就像访问文件一样。

现在我们来看客户端的情形。同样地，这里首先也必须使用 SOCKET 原语创建一个套接字，但是因为客户端使用什么地址对服务器而言无所谓，所以客户端不需要调用 BIND 原语。CONNECT 原语阻塞调用方，并主动发起建立连接过程。当 CONNECT 调用完成 (即接收到服务器发送过来的确认段)，客户进程被解除阻塞，于是连接就被建立起来。现在双方都可以使用 SEND 或者 RECV，在新建的全双工连接上发送或者接收数据。如果 SEND 和 RECV 调用不要求特殊选项的话，服务器或者客户也可以使用标准的 UNIX 系统调用 READ 和 WRITE。

在套接字模型中，连接的释放是对称的。当双方都执行了 CLOSE 原语之后，连接就被释放了。

套接字盛极一时，非常流行，并且已经成为抽象于应用层的传输服务的事实上标准。套接字 API 通常与 TCP 协议结合向用户提供了一种称为可靠字节流 (reliable byte stream) 的面向连接的服务，这是我们描述过的简单可靠比特管道。然而，其他协议也可以被用来实施这项服务，使用相同的 API，并且对传输服务用户来说应该是相同的。

套接字 API 的强大体现在应用程序可通过它使用其他的传输服务。举例来说，套接字可以与无连接传输服务结合使用。在这种情况下，CONNECT 设置远程传输对等实体的地址，SEND 和 RECEIVE 发送数据报给远程对等实体和接收来自远程对等实体的数据报 (通常还使用一组扩展调用，例如，SENDTO 和 RECEIVEFROM 强调报文，并且不限制应用只针对单个传输对等实体)。套接字也可与提供消息流而不是字节流的传输协议一起工作，而且可以进行拥塞控制，也可以不进行拥塞控制。例如，数据报拥塞控制协议 (DCCP, Datagram Congestion Controlled Protocol) 是一个带有拥塞控制机制的 UDP 版本 (Kohler 等, 2006)。传输层用户必须理解他们得到的是什么服务。

然而，套接字不太可能是传输接口上的最后一个字。例如，应用程序通常有一组相关的流，比如 Web 浏览器，它向同一台服务器请求多个对象。有了套接字，最自然契合应用程序的是为每个对象设立一个流。这种结构意味着拥塞控制可应用到每个数据流，而不是不甚理想地作用于一个组。这样的处理方式把管理包袱抛给了应用程序。为此必须制定新的协议和接口，使得应用程序更加有效并简单地管理一组相关流。其中的两个例子，分别是由 RFC 4960 定义的流控制传输协议（SCTP，Stream Control Transmission Protocol）和结构化流传输（SST，Structured Stream Transport）（Ford，2007）。这些协议必须略微修改套接字 API 才能获得把相关数据流组成一组的好处，它们也支持混合功能，比如把面向连接的流量和无连接流量混合在一起，甚至可以组合多条网络路径组合。时间会告诉我们它们能否获得成功。

6.1.4 套接字编程实例：Internet 文件服务器

作为一个真实套接字调用的基本事实例子，考虑图 6-6 中所示的客户端和服务端代码。这里我们有一个非常原始的 Internet 文件服务器，和一个使用该服务器的客户实例。这份代码有许多限制（后面将会提到），但原则上，服务器代码可以编译成二进制代码，并且可运行在任何连接到 Internet 的 UNIX 系统上。客户代码也可以编译，并且可运行在 Internet 任何地方的其他 UNIX 机器上。客户代码在执行的时候需要正确的参数，以便获取服务器机器上的任何文件，这些文件是服务器本身能够访问的。文件被送到标准输出，当然，可以将标准输出重定向到某一个文件或者一个管道。

```
/* A client program that can request a file from the server program.
 * The server responds by sending the whole file.
 */
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <netdb.h>

#define SERVER_PORT 12345      /* arbitrary, but client & server must agree */
#define BUF_SIZE 4096         /* block transfer size */

int main(int argc, char **argv)
{
    int c, s, bytes;
    char buf[BUF_SIZE];         /* buffer for incoming file */
    struct hostent *h;           /* info about server */
    struct sockaddr_in channel;  /* holds IP address */

    if (argc != 3) fatal("Usage: client server-name file-name");
    h = gethostbyname(argv[1]);  /* look up host's IP address */
    if (!h) fatal("gethostbyname failed");
```

图 6-6 使用套接字的客户端代码

```

s = socket(PF_INET, SOCK_STREAM, IPPROTO_TCP);
if (s < 0) fatal("socket");
memset(&channel, 0, sizeof(channel));
channel.sin_family = AF_INET;
memcpy(&channel.sin_addr.s_addr, h->h_addr, h->h_length);
channel.sin_port = htons(SERVER_PORT);

c = connect(s, (struct sockaddr *) &channel, sizeof(channel));
if (c < 0) fatal("connect failed");
/* Connection is now established. Send file name including 0 byte at end. */
write(s, argv[2], strlen(argv[2])+1);

/* Go get the file and write it to standard output. */
while (1) {
    bytes = read(s, buf, BUF_SIZE);    /* read from socket */
    if (bytes <= 0) exit(0);           /* check for end of file */
    write(1, buf, bytes);              /* write to standard output */
}
}

fatal(char *string)
{
    printf("%s\n", string);
    exit(1);
}

#include <sys/types.h>                /* This is the server code */
#include <sys/fcntl.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <netdb.h>

#define SERVER_PORT 12345             /* arbitrary, but client & server must agree */
#define BUF_SIZE 4096                /* block transfer size */
#define QUEUE_SIZE 10

int main(int argc, char *argv[])
{
    int s, b, l, fd, sa, bytes, on = 1;
    char buf[BUF_SIZE];               /* buffer for outgoing file */
    struct sockaddr_in channel;        /* holds IP address */

    /* Build address structure to bind to socket. */
    memset(&channel, 0, sizeof(channel)); /* zero channel */
    channel.sin_family = AF_INET;
    channel.sin_addr.s_addr = htonl(INADDR_ANY);
    channel.sin_port = htons(SERVER_PORT);

```

图 6-6 (续)

```

/* Passive open. Wait for connection. */
s = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP); /* create socket */
if (s < 0) fatal("socket failed");
setsockopt(s, SOL_SOCKET, SO_REUSEADDR, (char *) &on, sizeof(on));
b = bind(s, (struct sockaddr *) &channel, sizeof(channel));
if (b < 0) fatal("bind failed");

l = listen(s, QUEUE_SIZE); /* specify queue size */
if (l < 0) fatal("listen failed");

/* Socket is now set up and bound. Wait for connection and process it. */
while (1) {
    sa = accept(s, 0, 0); /* block for connection request */
    if (sa < 0) fatal("accept failed");

    read(sa, buf, BUF_SIZE); /* read file name from socket */
    /* Get and return the file. */
    fd = open(buf, O_RDONLY); /* open the file to be sent back */
    if (fd < 0) fatal("open failed");

    while (1) {
        bytes = read(fd, buf, BUF_SIZE); /* read from file */
        if (bytes <= 0) break; /* check for end of file */
        write(sa, buf, bytes); /* write bytes to socket */
    }
    close(fd); /* close file */
    close(sa); /* close connection */
}
}

```

图 6-6（续）

我们首先来看服务器的代码。在开始处它包含一些标准的头文件，其中最后 3 个头文件包含了一些主要的与 Internet 有关的定义和数据结构。接下来是一个宏定义，将 `SERVER_PORT` 定义成 12 345。这个数值是任意选取的。1024~65 535 之间的任何一个数值都可以正确地工作，只要不被其他进程使用；小于 1024 的数字为特权用户保留。

服务器代码中接下来两行定义了两个用到的常数。第一个常数决定了在文件传输过程中数据块的大小。第二个常数决定了允许多少个连接请求在排队等待处理，一旦连接请求到达这个数目以后，后续到达的连接请求将被丢弃。

在声明了局部变量之后，服务器代码开始执行。首先它对一个用来存放服务器 IP 地址的数据结构进行初始化。该数据结构很快被绑定到服务器的套接字中。`memset` 调用将这个数据结构初始化为全 0。紧随其后的 3 条赋值语句分别填充了该数据结构的 3 个字段。其中最后一个字段包含了服务器的端口。函数 `htonl` 和 `htons` 是必要的，它们将参数中的值转换成一种标准格式，所以，该服务器的代码既可以运行在 little-endian 字节序的机器（比如

Intel x86) 上, 也可以运行在 big-endian 字节序的机器 (比如 SPARC) 上。我们这里并不关心它们的确切语义。

接下来为服务器创建一个套接字, 并且检查是否出错 (由 `s<0` 标示)。在产品版本的服务器代码中, 错误消息可能包含更多的说明信息。调用 `setsockopt` 是必要的, 它允许这个端口可以被重复使用, 以便服务器能够永久地运行下去, 处理一个又一个请求。现在, 要绑定 IP 地址到套接字中, 然后检查调用 `bind` 是否成功。初始化过程中的最后一步是调用 `listen`, 这样就宣告了本服务器愿意接受入境呼叫, 并告诉系统当服务器在处理当前请求时如果有新的请求到来, 就将新请求挂起, 挂起等待处理的请求个数可以多达 `QUEUE_SIZE`。如果队列满了之后又有新的请求到来, 则直接将后来的请求丢弃即可。

到了这点, 服务器便进入了它的主循环, 这是一个永不退出的循环。终止服务器的唯一做法是从外部将服务器进程杀死。`accept` 调用阻塞服务器, 直到某个客户试图与它建立连接。如果调用 `accept` 成功, 则它返回一个套接字描述符, 利用该描述符可以进行读写操作, 就好像利用文件描述符从管道读写数据一样。然而, 与单向管道不同的是, 套接字是双向的, 所以, 既可以用 `sa` (接受的套接字) 从连接中读取数据, 也可以用它往连接上写数据; 而一个管道描述符可以用来读管道中的数据, 也可以往管道内写数据, 但不能同时读写。

当连接被建立之后, 服务器从连接中读取文件名。如果文件名不可用, 则服务器被阻塞住, 等待文件名的到来。服务器获得了文件名之后, 它打开该文件, 然后进入一个循环: 交替地从文件中读取数据块并且将数据块写到套接字中, 这个过程一直持续到整个文件被复制完毕为止。然后, 服务器关闭文件和连接, 并等待下一个连接的到来。它无限地重复这个循环。

现在让我们来看客户代码。为了理解客户代码的工作过程, 首先有必要理解客户被调用的方式。假设客户程序名为 `client`, 那么, 一个典型的调用如下:

```
client flits.cs.vu.nl /usr/tom/filename >f
```

只有当服务器已经在 `flits.cs.vu.nl` 机器上运行, 该机器上确实存在 `/usr/tom/filename` 文件, 并且服务器对该文件有读访问权限时, 上面这个 `client` 调用才能生效。如果调用成功, 那么, 服务器上的文件将通过 Internet 被传递给客户端, 并且写到客户端的本地文件 `f` 中, 然后客户程序退出。由于在一次传输之后, 服务器仍然在运行, 所以客户可以再次启动, 以获取其他的文件。

客户代码首先是一些包含语句和声明。开始执行时它先判断用户调用自己是否有正确的参数个数 (这里 `argc=3` 代表了程序名加上两个参数)。请注意, `argv[1]` 包含了服务器的名字 (比如 `flits.cs.vu.nl`), 通过 `gethostbyname` 把它转换为一个 IP 地址。这里的函数 `gethostbyname` 使用了 DNS 来查询机器名字对应的地址。我们将在第 7 章学习 DNS。

接下来创建一个套接字并执行初始化。之后, 客户调用 `connect` 函数, 企图与服务器建立一个 TCP 连接。如果在指定名字的机器上, 服务器已经启动运行, 并且被绑定到了 `SERVER_PORT` 端口, 那么在服务器当前空闲, 或者在 `listen` 队列中还有空间时, 连接 (最终) 将被建立起来。客户利用该连接可以将文件的名字发送过去, 做法很简单, 只要在套接字上执行写操作即可。发送的字节数是名字长度加 1, 必须在名字后面加一个字节 0 并

发送过去，服务器才能知道文件名在哪里结束。

现在客户进入了一个循环，它将文件从套接字中逐个数据块地读出来，再复制到标准输出上。完成这个过程之后，它就退出。

过程 `fatal` 打印出一条错误消息，然后退出程序。服务器代码也需要同样的过程，但是考虑到页面空间紧张，所以它被省略掉了。由于客户和服务器是单独编译的，而且通常运行在不同的计算机上，所以它们并不共享同一份 `fatal` 代码。

这两个程序（以及与本书相关的其他资料）可以从本书的网站获取：

<http://www.pearsonhighered.com/tanenbaum>

这台服务器在服务器领域不是最后的定论，纯粹为了记录而已。它的错误检查能力微薄，它的错误报告也很一般。由于它严格按顺序处理所有的请求（因为它只有一个线程），因此它的性能很差。它显然从来没有听说过安全性，并且使用裸露的 UNIX 系统调用并且没采用独立于平台的方式。同时它还做了一些在技术上是非法的假设，比如假设文件名一定适合缓冲区，而且文件传输以原子方式执行。尽管有这些缺点，但它仍然是一个可运行的 Internet 文件服务器。在本章的习题中，邀请读者对其进行改进。套接字编程的更多信息，请参阅（Donahoo, 2008）和（Calvert, 2009）。

6.2 传输协议的要素

传输服务由传输协议实现，两个传输实体之间的通信必须使用传输协议。传输协议在有些方面类似于我们在第 3 章中学习过的数据链路协议。这两种协议都要处理错误控制、顺序性和流量控制以及其他一些问题。

然而，两者之间也存在着重大的差别。这些差别是因为这两种协议的运行环境不同而造成的。如图 6-7 所示。在数据链路层，两台路由器通过一条有线或者无线物理信道直接进行通信；而在传输层，该物理信道被整个网络所替代。这种环境差异对于协议设计有很大的影响。

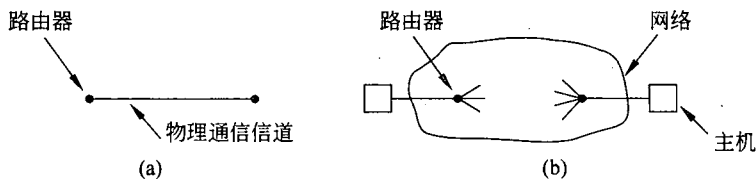


图 6-7
(a) 数据链路层环境；(b) 传输层环境

首先，在点到点链路上，无论是电缆或者光纤，路由器不必指定它要与哪一台路由器进行通话——每条出境线路直接通向一台特定的路由器。而在传输层，必须显式地指定接收方的地址。

其次，图 6-7 (a) 中，在一条线路上建立一个连接的过程非常简单：另一端总是在那里（除非它崩溃了才不在那里）。两边都不需要做很多事情。即使在无线链路上，建立过程也没有多大的不同，只要发出的消息足够到达所有的其他接收方。如果因发生错误而消息

没有被确认，可以再次重发。而在传输层中，初始的连接建立过程非常复杂，正如我们将会看到的那样。

数据链路层和传输层之间的另一个（非常恼人的）差别是，网络存在着潜在的存储容量。当路由器发送一帧到一条链路上后，该帧可能到达对方也可能丢失，但是它不可能先蹦跶一会儿，再躲到远处一个角落中，然后在其他数据包发送出去很久后突然又冒了出来。如果网络使用数据报技术，即网络内部的路由是独立进行的，那么就存在一个不可忽略的概率：一个包可能选取了风景优美的路线，一路观光姗姗来迟到达目的地，这将扰乱预期的接收顺序，甚至它的重复数据包都已经到达目的地它还没到呢。网络具有的这种延迟和重复数据包的特性所产生的后果有时是灾难性的，因此要求使用特殊的协议，以便正确地传输信息。

数据链路层和传输层之间最后一个差别是程度上的差别，而并非类别上的差别。这两层都需要缓冲和流量控制，但是，由于传输层上存在着大量并且数量可变的连接，而且由于连接之间的相互竞争造成连接的可用带宽上下波动，因此需要一种不同于数据链路层使用的方法。在第3章中讨论的有些协议为每条线路分配了固定数量的缓冲区，所以，当一帧到达时，总是有缓冲区可用。在传输层中，由于必须要管理大量的连接并且每个连接获得的带宽又是可变的，因此，为每条线路分配多个缓冲区的思路不再有吸引力。在本节中，我们将讨论所有这些重要的以及其他的问题。

6.2.1 寻址

当一个应用（比如一个用户）进程希望与另一个远程应用进程建立连接时，它必须指定要连接到哪个应用进程上（无连接的传输也有同样的问题：消息发送给谁？）。通常使用的方法是为那些能够监听连接请求的进程定义相应的传输地址。在 Internet 中，这些端点称为端口（port）。我们将使用通用术语传输服务访问点（TSAP, Transport Service Access Point）来表示传输层的一个特殊端点。网络层上的类似端点（即网络层地址）毫不奇怪地称为网络服务访问点（NSAP, Network Service Access Point）。IP 地址是 NSAP 的实例。

图 6-8 显示了 NSAP、TSAP 和传输连接之间的关系。应用进程（包括客户和服务端）可以将自己关联到一个本地 TSAP 上，以便与一个远程 TSAP 建立连接。这些连接运行在每台主机的 NSAP 之上，如图所示。在有些网络中，每台计算机只有一个 NSAP，但是可能有多个传输端点共享此 NSAP，因此需要某种方法来区分这些传输端点。

使用传输连接的一种可能场景如下所述。

(1) 主机 2 上的邮件服务器进程将自己关联到 TSAP 1522 上，等待入境连接请求的到来。至于进程如何将自己关联到 TSAP 上，这超出了网络模型的范畴，完全取决于本地操作系统。例如，用我们的 LISTEN 调用就可以做到这一点。

(2) 主机 1 上的应用进程希望发送一个邮件消息，所以它把自己关联到 TSAP 1208 上，并且发出一个 CONNECT 请求。该请求消息指定主机 1 上的 TSAP 1208 作为源，主机 2 上的 TSAP 1522 作为目标。这个动作最终导致在应用进程和服务端之间建立了一个连接。

(3) 应用进程发送邮件消息。

(4) 作为响应，邮件服务器表示它将传递该消息。

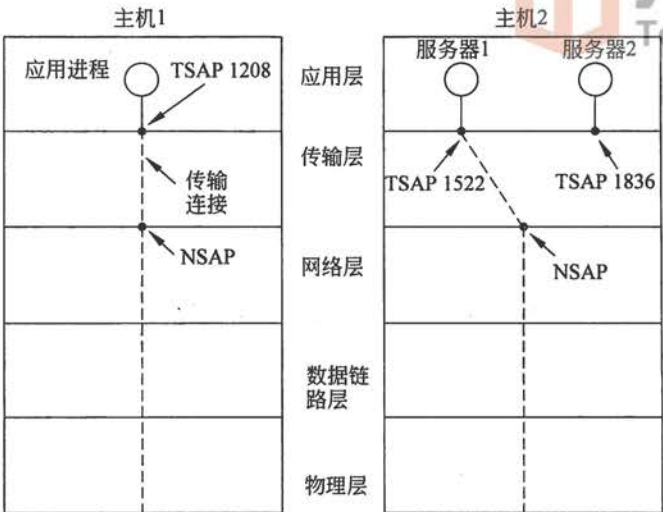


图 6-8 TSAP、NSAP 和传输连接

(5) 传输连接被释放。

请注意，在主机 2 上很可能还有其他的服务器被关联到其他的 TSAP 上，它们也在等待经过同一个 NSAP 到达的入境连接请求。

上面描绘的场景非常美好，除了我们已经把这个细微问题解决了：主机 1 上的用户进程如何知道邮件服务器被关联到了 TSAP 1522 上？一种可能是，很多年以来邮件服务器一直与 TSAP 1522 关联着，渐渐地所有的网络用户都知道了这个信息。在这个模型中，具有固定 TSAP 地址的服务被列出在一些知名的文件中，比如 UNIX 系统上的/etc/services 文件，该文件列出了哪些服务器被永久地关联到哪些端口上，事实上，我们可以发现邮件服务器被固定在 TCP 端口 25。

虽然固定的 TSAP 地址适用于少数永不改变的关键服务（比如 Web 服务器），但一般来说，用户进程经常需要跟其他用户进程通信，这些用户进程的 TSAP 地址无法预先得知，而且可能只存在较短的时间。

为了处理这些情形，通常考虑使用另一种方案。在这个方案中，存在一个称为端口映射器（portmapper）的特殊进程。为了找到一个给定服务名字（比如“BitTorrent”）相对应的 TSAP 地址，用户需要与端口映射器（它总是在监听一个知名的 TSAP）建立一个连接；然后，用户通过该连接发送一条消息指定它想要的服务名字；端口映射器返回相应的 TSAP 地址。之后，用户释放它与端口映射器之间的连接，再与所需的服务建立一个新的连接。

在这个模型中，当一个新的服务被创建时，它必须向端口映射器注册，把它的服务名字（通常是一个 ASCII 字符串）和 TSAP 告诉端口映射器。端口映射器将该信息记录到它的内部数据库中，所以，以后当用户查询时，它就知道答案了。

端口映射器的功能类似于电话系统中的查号操作员——提供的是从名字到电话号码之间的映射关系。如同在电话系统中一样，很重要的一点是端口映射器使用的知名 TSAP 地址一定是真正众所周知的。如果你不知道查号操作员的号码，那么你根本不可能呼叫查号

操作员来查询所需要的号码。如果你认为自己所拨打的信息服务号码是非常显然的，那么当你有机会到国外旅行的时候可以试一试。

一台机器上可能存在多个服务器进程，其中有许多服务很少被人使用。如果让这些服务器进程一直都活跃着，并整天监听一个稳定的 TSAP 地址，则是一种浪费。另一种方案的简化形式如图 6-9 所示，这个方案称为初始连接协议（initial connection protocol）。不是每个想象的服务器都在一个知名 TSAP 上监听，每台希望向远程用户提供服务的机器有一个特殊的进程服务器（process server）充当那些不那么频繁使用的服务器的代理。这个服务器在 UNIX 系统中称为 `inetd`。它在同一时间监听一组端口，等待连接请求的到来。一个服务的潜在用户发出一个连接请求，并指定他们所需服务的 TSAP 地址。如果该 TSAP 地址上没有服务器正等着，则他们得到一条与进程服务器的连接，如图 6-9（a）所示。

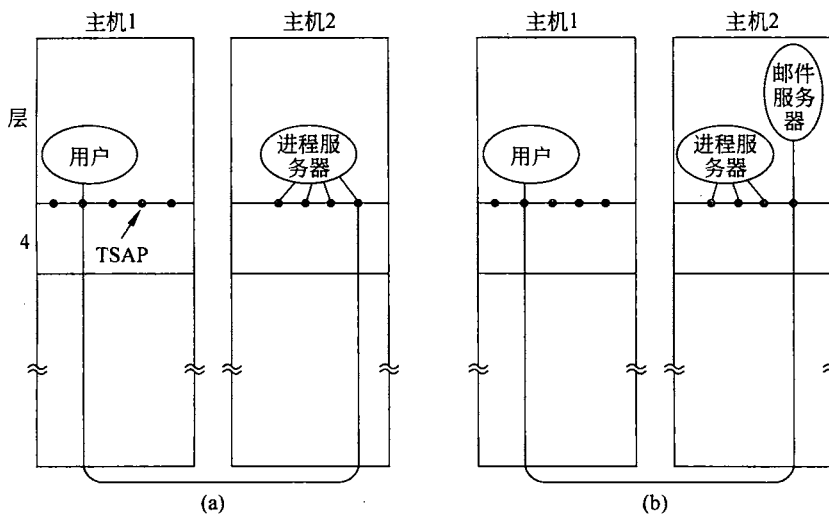


图 6-9 主机 1 上的一个用户进程是如何通过进程服务器与主机 2 上的邮件服务器建立连接的

在获取入境请求后，进程服务器派生出被请求的服务器，允许该服务器继承与用户的现有连接。新服务器完成要求的工作，而进程服务器可回去继续监听新的连接要求，如图 6-9（b）所示。这种方法只适用于服务器可按需创建的场合。

6.2.2 连接建立

建立一个连接听起来很容易，但是实际上却出奇的琐碎。初看起来，好像一个传输实体只要给接收方发送一个 `CONNECTION REQUEST` 段，然后等待 `CONNECTION ACCEPTED` 应答就足够了。当网络可能出现丢失、延迟、损坏和重复数据包的时候，问题就发生了。这些行为将导致极为严重的复杂性。

想象一个网络非常拥塞，以至于几乎所有的确认数据包都无法及时地返回到发送端，因此每个数据包都超时，因而被迫重传两次或者三次。假设该网络内部使用数据报技术，并且每个数据包遵循不同的路径。有些数据包可能受到网络流量拥挤的影响，从而需要很长时间才能到达目的地，也就是说，它们被网络延迟了，许久以后才被送到接收端，而此时发送端认为这些数据包已经丢失了。

可能出现的最坏噩梦是这样的情形：用户建立了一个与银行的连接，并且发送消息告诉银行把一大笔钱转移到一个并不是完全值得信赖的人的账户。不幸的是，数据包决定采取一条到目的地的风景路径，去探索网络的某个偏僻角落。然后，发送端超时，并且再次发送这些消息。这一次，数据包采取了一条最短路径，并且很快被交付给接收端，所以发送端释放连接。

不幸的是，最终原先的那批数据包终于从某个隐藏处冒了出来，并到达目的地，要求银行建立一个新的连接和汇款（再次）。银行没有办法得知这些是重复请求。它必须假设这是第二个并且独立的交易请求，因此再次转钱。

这种场景可能听起来似乎不可能发生，甚至令人难以置信，但问题就是这样：协议必须被设计成在所有情况下都能正确运行。在普通情况下协议的实现必须有效，以便取得良好的网络性能；但当出现罕见情况时协议也必须能够应付不会中断。如果协议做不到这点，则说明我们建立的一个公平的全天候网络竟然在一些条件下没有发出警告就失败了。

本节的其余部分，我们将研究延迟重复的问题，强调以可靠方式建立连接的算法，因此类似上面的梦魇不可能再发生。问题的症结在于被延迟的重复数据包在接收端看来是新数据包，从而造成重复接收。我们不能防止出现数据包的重复和被延迟的情况，但如果真的发生了这种情况，那么必须拒绝重复的数据包，不能把它们当做新数据包来处理。

这个问题可以有各种方式的解决途径，但没有一种令人非常满意。一种方法是使用一次性的传输地址。在这种方法中，每次传输都需要一个地址，而且是一个新产生的地址。当连接被释放时，该地址将被丢弃，并从来不被重复使用。这样，延迟的重复数据包将永远找不到抵达传输进程的途径，从而不会损害协议的正常运行。然而，这种方法使得首次与一个进程建立连接变得异常困难。

另一种可能的做法是连接发起方为每个连接分配一个唯一标识符（即一个序号，每建立一个连接，该序号就递增），并且将该标识符放在每个段中，也包括请求建立连接的那个段。当释放某个连接时，每个传输实体可以更新一张内部表，该表列出了所有过期的连接，形式为“对等传输实体，连接标识符”。每当到达一个连接请求，传输实体就检查该表，看它是否属于某一个以前已被释放掉的老连接。

不幸的是，这种方案有个基本缺陷：它要求每个传输实体无限期地维护一定数量的历史信息。这种历史信息必须在源机器和目标机器上保持一致。而且，如果一台机器崩溃，丢失了它的全部内存，那么它就不可能知道哪些连接标识符已经被对等实体用过了。

因此，我们需要采用不同的策略来简化问题。我们不再允许数据包在网络中无限期地生存下去，而是设计一种机制来杀死那些已经过时但仍在网络中蹒跚的数据包。有了这个限制，问题就变得好管理多了。

数据包的生存期可以用以下一种（或多种）技术被限定在一个给定的最大值之内：

- (1) 限制网络设计。
- (2) 在每个数据包中放置一个跳计数器。
- (3) 为每个数据包打上时间戳。

第一种技术包括任何一种避免数据包进入循环的方法，并结合某种限定延迟的方法，这个延迟要包括（已知的）最长可能路径上拥塞延迟的上界。第二种方法是，将跳计数器初始化为某个适当的值，然后每次数据包被转发的时候该跳计数器减一。网络协议简单丢

弃掉那些跳计数器变成零的数据包。第三种方法要求每个数据包携带它的创建时间，路由器负责丢弃那些年限超过某个预设值的数据包。后一种方法要求同步所有路由器的时钟，而这本身不是一件很容易完成的任务，实际上用跳计数器足够接近生存期。

实际上，我们不仅要保证一个数据包死亡，而且要保证它的所有确认最终也死亡，所以我们现在引入周期 T ，它是数据包实际最大生存期的某个不太大的倍数。最大数据包的生存期是一个网络的保守常数，对于 Internet 来说，它有点随意地取了 120 秒。而倍数与具体的协议有关，它只影响 T 的长短。如果在一个数据包被发送出去之后，等待了 T 秒时间，那么可以确信该数据包的所有痕迹现在都没有了，它和它的确认将来都不会突然冒出来而使问题复杂化。

限定了数据包生存期的上界之后，我们就有可能制定一种实际并万无一失的方法来拒绝重复的段。下面描述的方法来源于 (Tomlinson, 1975)，又被 (Sunshine 和 Dalal, 1978) 做了进一步的提炼。它的一些变种方法已得到广泛实际应用，包括 TCP。

方法的核心是源端用序号作为段的标签，使得该段在 T 秒内不被重用。 T 的大小以及数据包速率（数据包/秒）确定了序号的大小。这样，在任何给定的时间内只能出现一个给定序号的数据包。这个数据包的重复现象仍然有可能发生，但是它们必须被接收方丢弃。然而，这样的情况不会再发生：即一个具有相同序号的重复的老数据包击败一个新数据包，取而代之地被接收方接受。

为了解决一台机器崩溃之后丢失所有内存的问题，一种可能的解决方法是要求传输实体在机器崩溃之后空闲 T 秒。这段空闲期将确保所有老的段全部死掉，因而发送端可以启用任何一个序号值。然而，在复杂的互联网络中， T 可能会很大，因此这种策略不具吸引力。

相反，Tomlinson 建议每台主机都配备一个日时钟 (time-of-day clock)。不同主机上的时钟不需要同步。假定每个时钟均采用了二进制计数器的形式。该计数器以统一的时间间隔递增自己，而且，计数器的位数必须等于或者超过序号的位数。最后，也是最重要的假设是，即使主机停机时钟也不停下，它将持续不停地运行。

当建立一个连接时，时钟的低 k 位被用于同样是 k 位的初始序号。因此，与第 3 章中的协议不同的是，每个连接都从一个完全不同的初始序号开始对它的段进行编号。序号空间应该足够大，以便当序号回绕时，原来那些具有老序号的段都已经消失。时间和初始序号之间的这种线性关系如图 6-10 (a) 所示。禁止区域显示了段序号非法使用的对应时间。

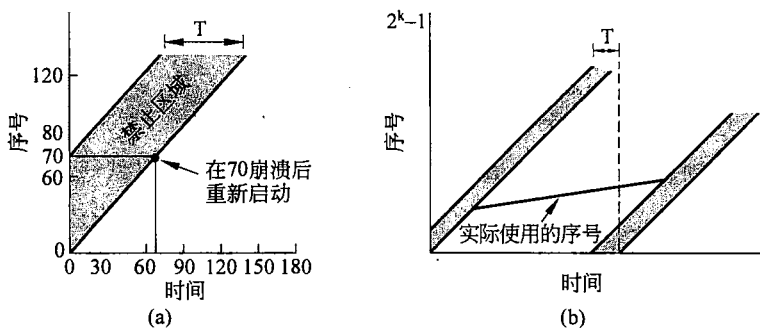


图 6-10

(a) 段不能进入的禁止区域; (b) 重新同步问题

任何发出去的段，如果其序号恰好落在禁止地区，那么它可能被延迟，因而假冒了其不久发出的具有相同序号的段。例如，如果主机在 70 秒时崩溃并重新启动，它将使用的初始序号基于它崩溃后的时钟，因此主机不可能启用一个位于禁止区域的较低序号。

一旦双方的传输实体已经统一了初始序号，则可以用任何一个滑动窗口协议来控制数据流的传输。窗口协议将发现并丢弃早就已经接受的重复数据包。事实上，初始序号曲线（图中的粗线）并不是线性的，而是一条梯状线，因为时钟是离散前进的。为了简化起见，我们忽略这一细节。

为了防止序号进入禁止区域，我们必须兼顾两个方面。协议可能以两种不同方式遇到麻烦。如果一台主机在一个新打开的连接上发送得太快，则实际序号与时间的曲线可能比初始序号与时间的曲线还要陡，导致序号进入禁止。为了防止这种情况出现，每个连接上的最大数据速率是每个时钟滴答一次发送一段。这同时也意味着在机器崩溃并重新启动之后，传输实体必须等待时钟滴答，才能打开一个新的连接，以免同样的序号被使用两次。这两点都倾向于使用短的时钟滴答（1 微秒甚至更短）。但是相对序号来说时钟不能滴答得太快。假设时钟速率为 C ，序号空间大小为 S ，则有 $S/C > T$ ，才能使得序号不至于回绕得太快。

由于发送速度太快而造成由下方往上进入禁止区域并不是带来麻烦的唯一情形。从图 6-10 (b) 我们可以看出，如果以任何低于时钟速率的数据率发送，实际使用的序号与时间之间的曲线最终会从左边进入到禁止区域中。实际序号曲线的斜度越大，则这种事件发生得越晚。为了避免这种情况发生，应该限制连接序号递增的速度不能太慢（或连接可能会持续多久）。

基于时钟的方法解决了无法区分重复段的问题。然而，要把这种方法应用到连接建立过程还要拔掉一个钉子。因为我们通常不会记住接收方在该连接上的序号，所以我们依然无法判断 CONNECTION REQUEST 段中包含的那个初始序号是否与当前连接的序号重复。这个问题在连接期间不会存在，因为滑动窗口能记住当前序号。

为了解决这个特殊问题，(Tomlinson, 1975) 引入了三次握手 (three-way handshake)。这是一个建立连接采用的协议，它要求一方检查连接请求是否的确是当前的。图 6-11 (a) 显示了主机 1 发起连接请求时的正常建立过程。主机 1 选择一个序号 x ，并且发送一个包含 x 的 CONNECTION REQUEST 段给主机 2。主机 2 回应一个 ACK 段作为对 x 的确认，并且宣告它自己的初始序号 y 。最后，主机 1 在它发送的第一个数据段中，对主机 2 选择的初始序号进行确认。

现在我们来看当出现延迟的重复控制段时三次握手协议是如何工作的。在图 6-11 (b) 中，第一个段是一个旧连接上被延迟了的重复 CONNECTION REQUEST。该段到达主机 2，而主机 1 对此并不知情。主机 2 对这个段的回应是给主机 1 发送一个 ACK 段，其效果相当于验证主机 1 是否真的请求建立一个新的连接。当主机 1 拒绝了主机 2 的连接建立请求后，主机 2 就意识到被一个延迟的重复段所欺骗了，于是放弃连接。这样，一个延迟的重复段没有造成任何伤害。

最坏的情况发生在延迟的 CONNECTION REQUEST 和 ACK 同时出现在网络中时。这种情形如图 6-11 (c) 所示。如同前一个例子一样，主机 2 得到一个延迟的 CONNECTION REQUEST，并回应了该请求。这里的关键是必须意识到，主机 2 已经建议使用 y 作为从主

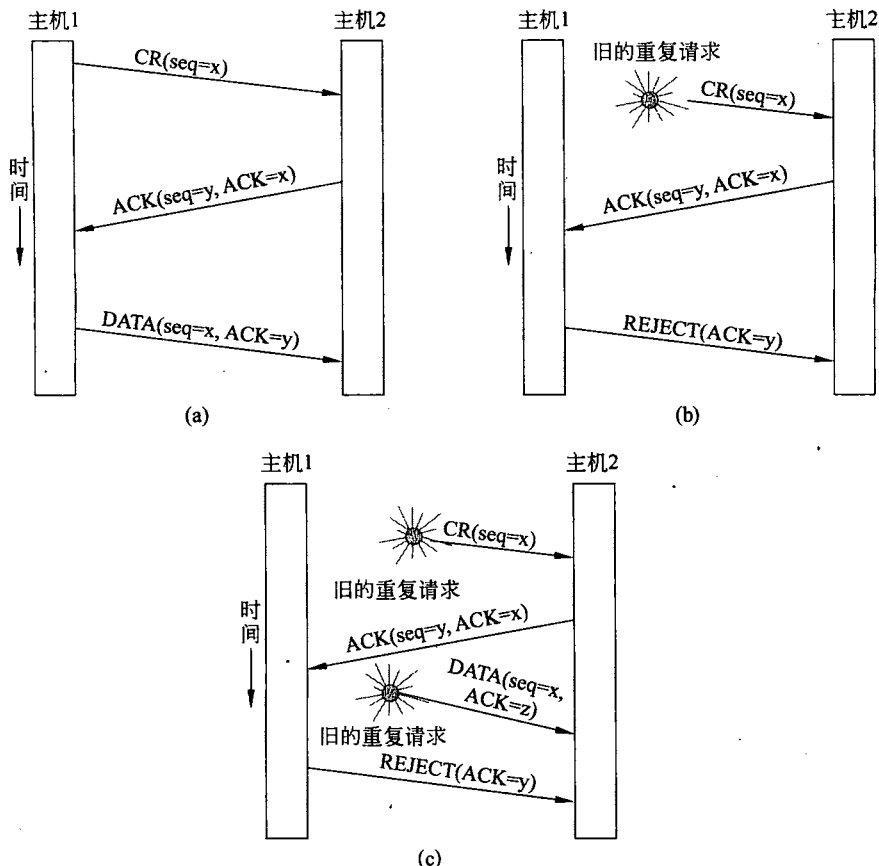


图 6-11 采用三次握手建立连接时的三种协议场景 (CR 表示 CONNECTION REQUEST)
(a) 正常操作; (b) 出现老的重复 CONNECTION REQUEST; (c) 重复 CONNECTION REQUEST 和重复 ACK

机 2 到主机 1 之间流量的初始序号, 同时也要知道, 现在已经没有包含序号为 y 的段或者对 y 的确认。当第二个延迟的段到达主机 2 时, 主机 2 注意到确认的是 z 而不是 y , 这个事实告诉主机 2 这也是一个老的重复数据包。在这里, 必须认识的一件非常重要的事情是, 老的段的任何组合都不能够让协议失败, 也不会出人意料地偶然建立一个连接。

TCP 使用三次握手机制来建立连接。在连接期间, 时间戳被用来辅助扩展 32 位序号, 以便它在最大数据包生存期间不会回绕, 甚至对于每秒千兆位的连接也一样。当 TCP 被用在越来越快的链路上时, 这种机制是对 TCP 的必要修复。该机制由 RFC1323 描述, 称为防止序号回绕 (PAWS, Protection Against Wrapped Sequence numbers)。在 PAWS 机制发挥作用以前, 整个连接期间对于初始序号, TCP 最初使用了刚才所描述的基于时钟的方案。然而, 结果显示竟然存在一个安全漏洞。时钟使得攻击者很容易预测到下一个初始序号, 然后发送数据包欺骗三次握手, 从而建立一个伪造的连接。为了堵住这个漏洞, 在实际使用中采用了伪随机的初始序号。然而, 仍然很重要的是一个间隔内初始序号不能重复, 即使它们随机出现。否则, 延迟的重复数据包可能肆虐成灾。

6.2.3 连接释放

释放一个连接要比建立一个连接容易得多。然而，这其中也存在着许多让人意想不到的陷阱。正如我们前面提到过的，终止连接的方式有两种：非对称释放和对称释放。非对称释放连接是电话系统的工作方式：当一方挂机后，连接就被中断了。对称释放连接是把连接看成两个独立的单向连接，要求单独释放每一个单向连接。

非对称释放方法太冒失，可能导致数据的丢失。请考虑图 6-12 中的场景。当连接被建立之后，主机 1 发送一段，它正确地到达了主机 2；然后，主机 1 发送另一段。不幸的是，主机 2 在第二个段到达之前发出了 DISCONNECT。结果该连接被释放，而数据却丢失了。

很显然，我们需要一个更加复杂的释放协议来避免数据丢失。一种方法是使用对称释放方式，每个方向被单独释放，两个方向的连接释放相互独立。因而，即使当主机发送了 DISCONNECT 段以后，它仍然可以接收数据。

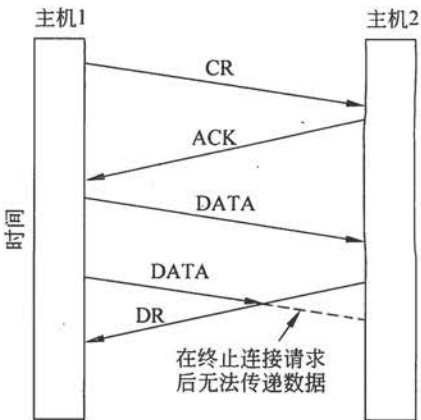


图 6-12 突然中止连接导致数据丢失

当连接两端的每个进程有固定数量的数据要发送，并且清楚地知道何时发送完这些数据时，对称释放方法可以断开连接。在其他的情形中，要确定所有的工作都已完成并且连接应该被终止并不是那么显而易见的容易。你可以想象这样一个协议，主机 1 说“我已经完成任务了，你也完成了吗？”如果主机 2 回答：“我也完成了。再见，可以安全地释放连接了。”

不幸的是，这个协议并不总能正确地工作。有一个著名的问题说明了这点。它就是所谓的两军对垒问题（two-army problem）。请想象一支白军被围困在一个山谷中，如图 6-13 所示。两旁的山上都是蓝军。白军的实力超过了两旁任何一支蓝军单独的力量，但是两支蓝军合起来的实力却超过了白军。如果任何一支蓝军单独发起进攻，则它将被白军击败；然而，如果两支蓝军同时发起攻击，则它们将会取得胜利。



图 6-13 两军对垒问题

两支蓝军希望能够同时发动攻击。然而，它们唯一的通信介质是派士兵穿过山谷传递

消息，而在穿越山谷时士兵可能会被白军抓住，从而丢失消息（即它们必须使用一条不可靠的通信信道）。现在的问题是：是否存在一个让蓝军获胜的协议？

假设蓝军 1 号的指挥官发送这样一条消息：“我建议我们在 3 月 29 日的黎明时分发起进攻。怎么样？”现在假设该消息到达了蓝军 2 号，2 号指挥官同意这一建议，并且他的回信安全地回到了蓝军 1 号。进攻会如期进行吗？可能不会，因为蓝军 2 号指挥官不知道他的回信是否能送到。如果回信没有送过去，蓝军 1 号将不会发动进攻，所以对他来说，贸然发动进攻将是十分愚蠢的。

现在我们对协议进行改进，将它变成一个三次握手协议。最初建议的发起方必须对应答消息进行确认。假设没有消息丢失，蓝军 2 号将得到确认；但是，蓝军 1 号指挥官现在犹豫了。毕竟，他不知道他的确认信是否送过去了，如果确认信没有送到蓝军 2 号，他知道蓝军 2 号就不会发动进攻。我们当然可以设计一个四次握手协议，但是，这并不能帮助我们解决上面任何一个问题。

实际上，可以证明完成这一任务的协议并不存在。我们用反证法，假设存在某一个协议可以正确工作。该协议的最后一条消息可能至关重要，也可能不那么重要。如果不重要，则从协议中去掉这条消息（以及其他所有无关紧要的消息），这样我们得到的协议中每条消息都是至关重要的。如果最后一条消息没有被送过去，则情形会怎么样呢？我们刚才说过了它是至关重要的，所以如果它丢失了，则进攻就不会如期进行。由于最后一条消息的发送端永远也无法确定它是否正确地到达，所以他不会冒险发动进攻。更糟的是，另一支蓝军也明白这一点，所以它也不会发动进攻。

为了看清楚两军对垒问题与释放连接问题之间的相关性，只要用“断开连接”来代替“发动进攻”就可以。如果任何一方要在确定另一方做好了断开连接的准备之后才断开连接，那么，断开连接的操作将永远也不可能发生。

实际上，我们可以把这种双方协商释放连接的需求推给传输层的用户，使得连接两端独立决定是否释放连接，从而避免出现上述窘态。由此这个问题很容易地就解决了。图 6-14 显示了使用三次握手法释放连接的 4 个场景。虽然这个协议并非完全没有错误，但是通常情况下已经足够了。

在图 6-14 (a) 中，我们看到正常情况下，一个用户发送一个 DR (DISCONNECTION REQUEST) 段来启动释放连接过程。当该段到达对方，接收端也发回一个 DR 段，并启动一个计时器，设置计时器的目的是为了防止它的 DR 丢失。当这个 DR 到达时，最初的发送端发回一个 ACK 段，并且释放连接。最后，当 ACK 返回后，接收端也释放连接。释放一个连接意味着传输实体将有关该连接的信息从它的内部表（记录了所有当前已打开的连接）中删除，并且通知该连接的所有者（传输用户）。这个动作与传输用户发出一个 DISCONNECT 原语有所不同。

如果最后的 ACK 段被丢失，则如图 6-14 (b) 所示，这种情形可以通过一个计时器来补救。当计时器超时，无论如何连接都要被释放。

现在考虑第二个 DR 被丢失的情形。发起释放连接的用户接收不到期望的响应，所以它将超时，于是再次尝试释放连接。在图 6-14 (c) 中，我们可以看到这个工作过程，图中假定第二次过程中没有段丢失，所有的段都被及时地递交给正确的接收端。

图 6-14 (d) 所示的最后一个场景与图 6-14 (c) 中相同，但由于丢失段的原因，所有

重传 DR 的尝试都失败。经过 N 次重试之后，发送端放弃了，并且释放连接。同时，接收端超时，于是退出连接。

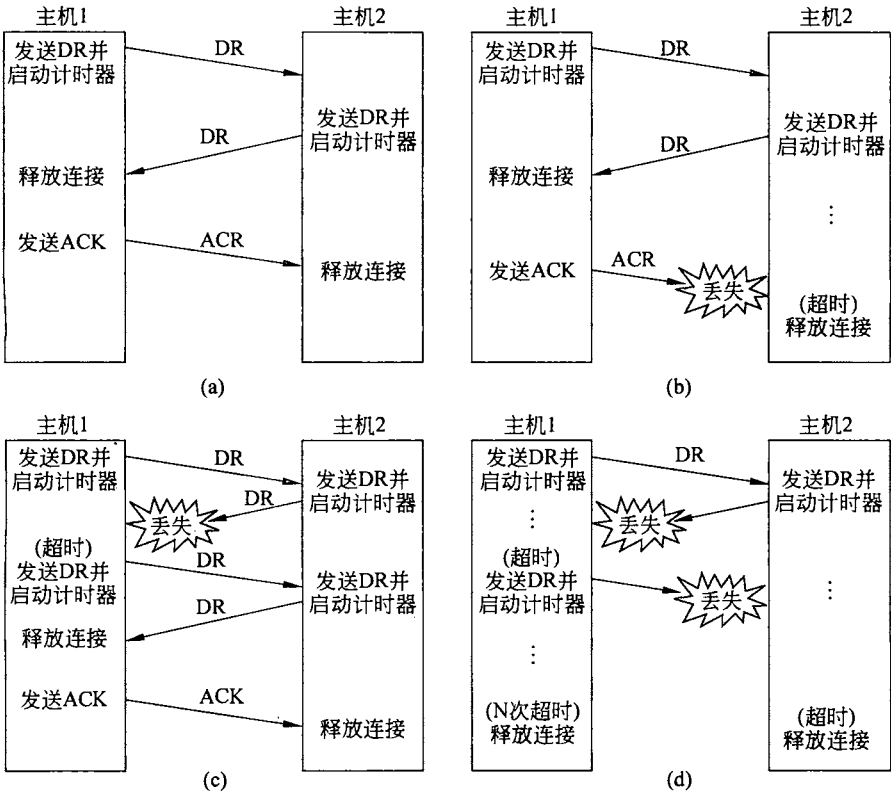


图 6-14 释放连接的 4 种协议场景

(a) 正常的三次握手情况；(b) 最后一个 ACK 被丢失了；(c) 响应丢失了；(d) 响应和随后的 DR 都丢失

虽然这个协议在一般情况下已经足够了，但在理论上，如果初始的 DR 和 N 次重传全部丢失，则协议可能失败。发送端将最终放弃发送释放连接请求并强行释放连接，而另一方对所有的释放连接尝试一无所知，它的连接仍然处于活跃状态。这种情况会导致一个半开的 (half-open) 连接。

如果我们不允许发送端在经过 N 次重试之后放弃，而是让它不停地重试，直至得到对方的响应，那么，这个问题就可以避免。然而，如果另一方允许超时，那么发送端将真的要永远尝试下去，因为没有响应会出现。如果我们不允许接收端超时，那么图 6.14 (d) 中的协议将被悬挂起来。

杀死半开连接可以采用这样一条规则：如果在规定的一段时间内没有段到来，该连接将被自动断开。以这种方式，如果一方断开连接，则另一方将检测到该连接没有活动，于是也会将其释放。这条规则也需要考虑连接中断的情况（由于网络不再在主机之间传送数据包），此时连接两端谁都没有断开连接。当然，如果引入这条规则，那么每当传输实体发送一个段的时候，它必须要先停止一个计时器，然后再重新启动。如果该计时器超时，则传送一个哑段，其目的仅仅是为了避免另一方断开此连接。另一方面，采用了自动断开连接规则后，如果一个空闲连接上多个连续的哑段全部丢失，则首先是一方然后是另一方都

将自动断开连接。

我们不再进一步讨论有关释放连接的问题了，但是现在你应该很清楚，释放一个连接而不会丢失数据并不像初看起来那么简单。这里传达的经验是，传输层用户必须参与进来决定何时断开连接——这个问题传输实体本身无法彻底地解决。要看清楚应用程序在这里的重要性，考虑这样的情形：虽然 TCP 通常采用对称释放方式来断开连接（每一边在发送完自己的数据之后用一个 FIN 数据包独立关闭其一半的连接），许多 Web 服务器给客户端发送一个 RST 包，导致突然关闭连接，这种工作方式更像非对称释放方式。之所以 Web 服务器能这样处理，是因为它知道数据交换模式。首先，它接收来自客户端的请求，这是客户端发送的所有数据；然后它给客户端发送一个响应。当 Web 服务器完成其响应工作后，意味着两个方向上的所有数据都已经发送完毕。服务器可以给客户端发送一个警告，然后骤然关闭连接。如果客户端得到这样的警告，它会释放它的连接状态；如果客户端没有得到警告，最终它会意识到服务器不再与自己交谈，并释放连接。在这两种情况下数据都已经得到成功传送。

6.2.4 差错控制和流量控制

在详细讨论了连接的建立和释放过程以后，现在我们来看如何在使用连接的过程中进行管理。关键问题是差错控制和流量控制。差错控制确保数据传输具备所需的可靠性，通常指所有的数据均被无差错地传送到目的地。流量控制是防止快速发送端淹没慢速接收端。

这两个问题我们在数据链路层已经考查过。传输层采用的解决方案与我们在第 3 章学过的一样。这里再简单回顾一下：

(1) 帧中携带一个检错码（比如，CRC 或者校验和）用于检测信息是否被正确接收。

(2) 帧中携带的序号用于标识本帧，发送方在收到接收方成功接收后返回的确认之前，必须重发帧。这种机制称为自动重复请求（ARQ, Automatic Repeat reQuest）。

(3) 任何时候允许发送方发送一定数量的帧，如果接收方没有及时返回确认，则发送方必须暂停。如果只允许发送一帧，则协议称为停等式（stop-and-wait）协议。较大的窗口可使得发送管道化，因而提高距离长且速度快的链路性能。

(4) 滑动窗口（sliding window）协议结合了这些功能，还能被用于支持数据的双向传送。

鉴于这些机制已成功应用于链路层上的帧，很自然地人们会怀疑它们是否也适用于传输层上的段。然而，实际上链路层和传输层的重复不多。即使使用相同的机制，它们在功能上和程度上都有很大的差异。

首先是功能上的差异，考虑错误检测。链路层的校验和保护一个穿过单条链路的帧。传输层的校验和则保护跨越整个网络路径的段，这是一个端到端的校验机制，与每条链路上的校验并不相同。（Saltzer 等，1984）描述了一种场景，数据包在一个路由器内部被损坏的情形。链路层校验和只保护在一条链路上经过的数据包，而没有考虑它们在路由器内部出错的情况。因此，即使根据每条链路的校验和检查都正确，数据包仍然会被不正确地传递。

Saltzer 等给出的这个例子和其他的例子阐明了端到端的观点。根据这种观点，执行端

到端的传输层校验对传输的正确性至关重要，而链路层的校验反而不是必不可少，但也仍然有价值，因为能提高性能（如果没有链路层的校验，损坏的数据包就必须沿整条路径重传，这显然是没有必要的）。

至于传输层和数据链路层在程度上的差异，考虑重传和滑动窗口协议。对于大多数无线链路，卫星链路除外，发送方一次只能发送一帧。也就是说，链路的带宽延迟乘积很小，甚至不能在链路上容纳一个完整的帧。在这种情况下，一个小的窗口足够产生良好的性能。例如，802.11 使用停-等式协议，每个帧被传输或者重传，直到它被接收方确认窗口才能移动到下一帧。大于一帧的窗口不仅没有改善协议性能，而且增加了控制复杂性。对于有线和光纤链路，比如（交换式）以太网或 ISP 骨干网，链路的误码率很低，因而可以忽略重传，因为端到端的重传会修复剩余的那些被链路层丢失的帧。

另一方面，许多 TCP 连接的带宽延迟乘积远远大于单个段。考虑美国的一个连接，发送速率为 1 Mbps，往返时间为 100 毫秒。即使这样一个慢速连接，在发送端发送一段并接收确认所需要的这段时间内，已经在接收端存储了 200 Kb 的数据。这种情况下，必须使用一个大的滑动窗口。停-等式的控制会削弱协议性能。在我们的例子中，它会限制性能，无论实际网络多快，总是每 200 毫秒发送一帧，或者说每秒发送 5 段。

既然传输协议通常使用较大的滑动窗口，我们将着眼于更仔细的缓冲数据问题。由于主机可能有多个连接，每个连接单独处理，因此它可能需要一个相当数量的缓冲区用作滑动窗口。发送端和接收端两端都需要缓冲区。当然，发送端需要用这些缓冲区来保存所有已经传输但尚未得到确认的段。之所以发送端需要缓冲区是因为发送的段可能会丢失，必须重传，因此要保存已发但未确认的段。

然而，由于发送端缓冲着发送的段，接收端可能会或可能不会为特定的连接设置专用的特定缓冲区，具体按它认为合适的方式做。例如，接收端可能只设立一个缓冲池让所有连接共享。当一个段到达接收端，接收端企图动态获得一个新缓冲区。如果刚好有个缓冲区可用，则该段被接受，否则将被丢弃。由于发送端时刻准备重新传输网络中丢失的段，因此接收端丢弃段并没有造成永久的伤害，虽然要浪费一些资源。发送端不断尝试发送，直到它得到确认为止。

源缓冲区和目标缓冲区之间的最佳权衡取决于由连接承载的流量类型。对于低带宽的突发流量（比如交互式终端产生的流量），不设置任何缓冲区是合理的；相反，如果段肯定偶尔会被丢弃，那么在连接两端动态获取缓冲区，具体的做法依赖于发送端的缓冲。另一方面，文件传输和其他高带宽流量，如果接收端有专用的满窗口的缓冲区最好，这样能允许数据流以最大速率发送。这就是 TCP 使用的策略。

还有一个问题涉及如何组织缓冲池。如果大多数段的长度都差不多，很自然的做法是将缓冲区组织成一个由大小统一的缓冲区构成的池，每个缓冲区容纳一个段，如图 6-15 (a) 所示。然而，如果段长度的差异范围很大，可能是一个请求 Web 网页的短数据包，也可能是 P2P 文件传输中的大数据包，那么，用固定长度的缓冲区池就有问题。如果将缓冲区设置成最大可能的段长，那么当短数据包到来时就会浪费空间；如果将缓冲区设置成比最大的段要小，则长的段就需要多个缓冲区，从而带来额外的复杂性。

解决缓冲区大小问题的另一个方法是使用可变大小的缓冲区，如图 6-15 (b) 所示。这里的优点是能获得更好的内存利用率，付出的代价是缓冲区的管理更加复杂。第三种可能

的方案是为每个连接使用一个大的循环缓冲区，如图 6-15 (c) 所示。这个系统古朴典雅，并且不依赖于段的大小，但只有当连接重载时，内存的使用情况才很好。

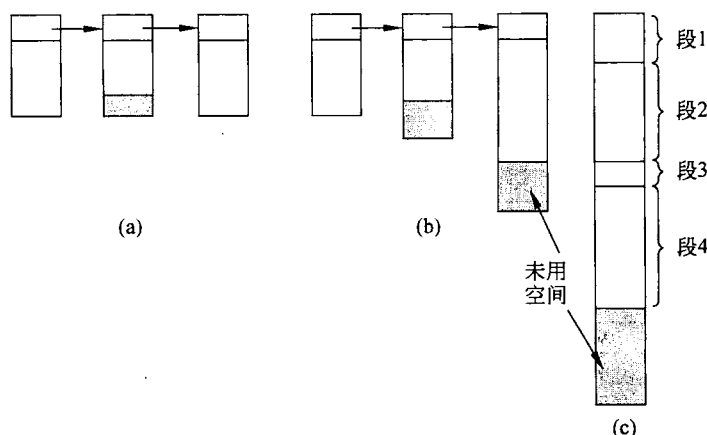


图 6-15

(a) 链式固定大小的缓冲区；(b) 链式可变大小的缓冲区；(c) 每个连接一个大循环缓冲区

随着连接被打开和关闭，以及流量模式的变化，发送端和接收端需要动态地调整它们的缓冲区分配策略。因此，传输协议应该允许发送主机请求另一端的缓冲区空间。缓冲区可以分配给每个连接，或者综合分配给两台主机之间当前正在运行的所有连接。另一种做法是，接收端知道自己的缓冲区情况（但是不知道流量的情况），可以告诉发送端“我已经为你预留了 X 个缓冲区。”如果打开的连接数量增加，或许有必要减少为每个连接分配的缓冲区数。因此，协议应该提供这种协商能力。

为了管理动态缓冲区的分配，一种合理的惯常做法是将缓冲与确认机制分离，这种做法与第 3 章介绍的滑动窗口协议做法不同。动态的缓冲区管理实际上意味着一个可变大小的窗口。初始时，发送端根据它期望的需求，请求一定数量的缓冲区。然后，接收端根据它的能力分配尽可能多的缓冲区。每次发送端传输一段，它必须减小分配给它的缓冲区数，当分配给它的缓冲区数达到 0 时，完全停止发送。然后，接收端在逆向流量中捎带上单独的确认和缓冲区数。TCP 采用这种模式，将缓冲区的分配捎带在头的 Window size 字段中。

图 6-16 显示了一个数据报网络如何动态管理窗口的例子，其中序号用 4 位标识。在例子中，段的数据流从主机 A 发往主机 B，段的确认和缓冲区分配流逆向传输。初始时，A 想要 8 个缓冲区，但是，B 只分配了 4 个；然后 A 发送 3 个段，其中第 3 个段被丢失。在第 6 行，B 确认已经接收到直至（含）序号 1 的所有段，从而允许 A 释放这些缓冲区；同时进一步通知 A，允许发送从序号 1 之后开始的 3 个段（即段 2、3 和 4）。A 知道自己已经发送了段 2，所以它认为现在可以发送段 3 和段 4，于是它接下去就这么做了。此刻，A 被阻塞，它必须等待分配更多的缓冲区。然而，在阻塞过程中，因超时而导致的重传仍然可以进行（第 9 行），因为重传帧所用的缓冲区早已被分配。在第 10 行，B 确认已经接收到直至（含）序号 4 的所有段，但是拒绝让 A 继续发送。这样的情形对于第 3 章中的固定大小窗口协议是不可能发生的。下一个从 B 到 A 的段表明 B 已分配了另一个缓冲区，从而允许 A 继续发送。当 B 有缓冲空间，并很可能 B 的传输用户接受了更多的段数据时，就会发生这种情况。

	主机A	消息	主机B	注释
1	→	< request 8 buffers>	→	A 需要8个缓冲区
2	←	<ack = 15, buf = 4>	←	B 只同意接收消息0~3
3	→	<seq = 0, data = m0>	→	A 现在剩3个缓冲区
4	→	<seq = 1, data = m1>	→	A 现在剩2个缓冲区
5	→	<seq = 2, data = m2>	...	消息丢失, 但A 认为还剩1个
6	←	<ack = 1, buf = 3>	←	B 确认0和1, 允许消息2~4
7	→	<seq = 3, data = m3>	→	A 有1个缓冲区
8	→	<seq = 4, data = m4>	→	A 耗尽缓冲区, 必须停止
9	→	<seq = 2, data = m2>	→	A 超时, 并且重传
10	←	<ack = 4, buf = 0>	←	B 确认了所有的消息, 但A 仍然阻塞着
11	←	<ack = 4, buf = 1>	←	A 现在可以发送消息5
12	←	<ack = 4, buf = 2>	←	B 从某处找到了1个新缓冲区
13	→	<seq = 5, data = m5>	→	A 还剩下1个缓冲区
14	→	<seq = 6, data = m6>	→	A 现在被再次阻塞
15	←	<ack = 6, buf = 0>	←	A 仍旧被阻塞着
16	...	<ack = 6, buf = 4>	←	潜在的死锁

图 6-16 动态缓冲区分配
箭头显示传输的方向。省略号指出丢失一个段

在数据报网络中, 如果控制段可能丢失(肯定会丢), 则这种缓冲区分配方案有可能引发一些潜在的问题。请看第 16 行, B 现在为 A 分配了更多的缓冲区, 但是, 这个缓冲区分配的段丢失了。由于控制段没有序号, 也不会超时, 所以 A 现在死锁了。为了避免出现这种情况, 每台主机应该定期地在每个连接上发送控制段, 这些控制段给出确认和缓冲区状态。采用这种方法, 死锁迟早会被打破。

到现在为止, 我们已经心照不宣地假设对发送端数据传输速率的唯一限制在于接收端的可用缓冲空间量。情况往往并非如此。内存价格曾经很昂贵, 但目前已大幅下降。主机可配备足够的内存, 因此缺乏缓冲区已经不是个问题; 如果真是个问题, 也只是针对广域连接。当然, 这取决于缓冲区是否被设置得足够大, 并非总是 TCP 的情况(zhang 等, 2002 年)。

当缓冲区空间不再限制最大数据流时, 另一个“瓶颈”将会出现: 网络的承载能力。如果相邻路由器之间每秒至多交换 x 个数据包, 并且在一对主机之间有 k 条互不相交的路径, 那么, 不管每条连接的两端有多少可用的缓冲空间, 它们每秒交换的段不可能超过 kx 个。如果发送端推进得太快(即发送速率超过 kx 段/秒), 网络将变得拥塞, 因为它不可能以超过段的入境速度来递交这些段。

这里需要的是一种基于网络承载容量而不是接收端缓冲容量的机制。(Belsnes, 1975) 建议使用一种滑动窗口流量控制方案。在他的方案中, 发送端动态地调整窗口的大小, 以便与网络的承载容量相匹配。这意味着动态滑动窗口可以同时实现流量控制和拥塞控制。如果网络每秒钟能够处理 c 个段, 并且往返时间是 r (包括发送、传播、排队、接收端的处理以及确认的返回), 那么, 发送端的窗口应该是 cr 。使用这样的窗口, 发送端通常可以按流水线方式全速地运行。此时, 网络性能只要减小一点点就有可能导致发送端被阻塞。因为每个给定流的可用网络容量随时在变, 因此窗口大小也应该频繁地做出调整来跟踪当前网络承载能力的变化。正如我们后面将看到的那样, TCP 就采用了类似的控制策略。

6.2.5 多路复用

多路复用（或多个会话共享连接、虚电路和物理链路）在网络体系结构的不同层次发挥着作用。在传输层，有几种方式需要多路复用。例如，如果主机只有一个网络地址可用，则该机器上的所有传输连接都必须使用这个地址。当到达了一个段，必须有某种方式告知把它交给哪个进程处理。这种情况，称为多路复用（multiplexing），如图 6-17（a）所示。在图中，4 个独立的传输连接都使用了相同的网络连接（即 IP 地址）到达远程主机。

多路复用对传输层之所以非常有用，还存在另一个原因。例如，假设一台主机有多条网络路径可用。如果用户需要的带宽和可靠性比其中任何一条路径所能提供的还要多，那么一种解决办法是以轮询的方式把一个连接上的流量分摊到多条网络路径，如图 6-17（b）所示。这种手法称为逆向多路复用（inverse multiplexing）。若打开了 k 条网络连接，则有效带宽可能会增长 k 个因子。逆向多路复用的一个例子是流控制传输协议（SCTP, Stream Control Transmission Protocol），它可以把一条连接运行在多个网络接口上。相反，TCP 使用了单个网络端点。逆向多路复用也可应用在链路层，把几条并发运行的低速率链路当作一条高速率链路使用。

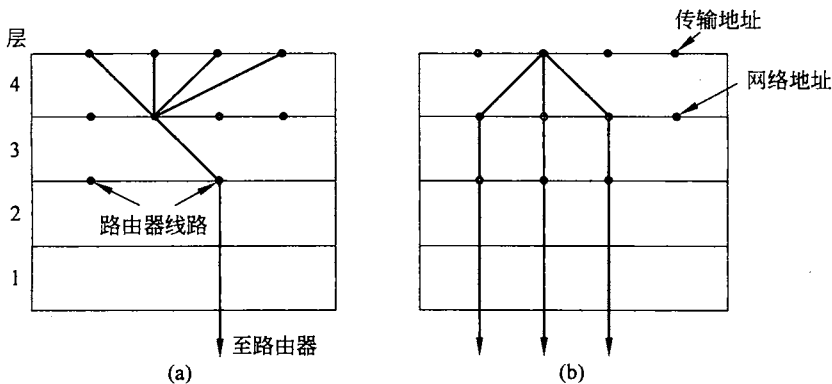


图 6-17
(a) 多路复用; (b) 逆向多路复用

6.2.6 崩溃恢复

如果主机和路由器崩溃，或者连接持续时间较长（比如正下载一个大软件或媒体文件），那么，如何从这些崩溃事件中恢复运行就成为一个问题。如果传输实体完全位于主机内，则从网络和路由器的崩溃中恢复比较直截了当。传输实体总是在期待着丢失的段，而且它们知道通过重传来处理这些段的丢失。

一个更加麻烦的问题是如何从主机的崩溃中恢复过来。尤其是，当服务器崩溃并且快速启动之后，客户可能期望还能够继续工作。为了说明其中的困难程度，我们假设一台主机（客户）使用一个简单的停-等式协议，正在给另一台主机（文件服务器）发送一个大文件。服务器上的传输层只是简单地将入境段逐个传递给传输用户。在传输到一半时，服务器崩溃了。当服务器恢复运行以后，它的内部表被重新进行了初始化，所以，它根本不知

道原来的传输过程进行到了哪里。

为了试图从原先的状态中恢复过来，服务器可以给所有其他主机发送一个广播段，宣告自己刚才崩溃了，并要求它的客户们告知关于所有打开连接的状态信息。每个客户可能处于以下两种状态之一：发出一个段，但尚未确认，S1 状态；或者，没有未完成的段，S0 状态。根据这一状态信息，客户必须确定是否重传最近的段。

初看起来似乎很明显：当客户知道服务器崩溃时，当且仅当有一个未确认的段悬挂着时（即处于状态 S1）它才重传该段。然而，进一步探究这种天真的做法便会发现其中的困难。例如，请考虑这样的情形：服务器的传输实体首先发送一个确认，然后在确认被发出以后再执行写操作，把数据传给应用进程。将一个段写到输出流和发送一个确认，这是两件不能同时进行的独立事件。如果在确认被发送出去之后，但是在写操作完成之前服务器崩溃了，那么客户将会接收到确认；当服务器崩溃恢复之后发出的广播消息到来的时候，该客户处于状态 S0。因此客户不会重传段，因为它（不正确地）认为段已经到达服务器了。客户的这一决定将导致错失一个段。

此刻你可能会这样想：“这个问题很容易解决，只要重新修改传输实体的程序，让它先执行写操作再发送确认即可。”你可以再试一试。请想象一下，如果写操作已经完成，但是在发送确认之前服务器崩溃了，那么，客户将处于状态 S1 中，因此它会重传，从而导致在服务器应用进程的数据流中出现一个未被检测出来的重复段。

无论如何编写客户和服务器的代码，总是存在使协议无法正确恢复运行的情形。服务器程序可以编成两种方式：先发送确认，或者先写数据。而客户程序可以编成四种不同方法：总是重传最后一段、永远不重传最后一段、仅当在状态 S0 时才重传，或者仅当在状态 S1 时才重传。服务器和客户两者结合起来有 8 种操作组合，但是正如我们将会看到的，对于每一种组合都存在一些使协议失败的事件。

服务器端可能发生 3 种事件：发送一个确认（A）、将数据写到输出进程（W）和崩溃（C）。这 3 种事件的发生有 6 种不同的顺序：AC（W）、AWC、C（AW）、C（WA）、WAC 和 WC（A），这里的括号表示 A 或者 W 不可能发生在 C 的后面（即一旦崩溃那就彻底崩溃）。图 6-18 显示了客户和服务器策略的 8 种组合，以及每一种组合的有效事件序列。请注意，对于每一种策略，总是存在使协议失败的事件序列。例如，如果客户总是重传的话，则 AWC 事件将生成一个无法检测到的重复，即使其他两个事件可以正确地工作。

发送主机采用的策略	接收主机采用的策略					
	先ACK，后写			先写，后ACK		
	AC(W)	AWC	C(AW)	C(AW)	WAC	WC(A)
总是重传	OK	DUP	OK	OK	DUP	DUP
永远不重传	LOST	OK	LOST	LOST	OK	OK
在状态S0时重传	OK	DUP	LOST	LOST	DUP	OK
在状态S1时重传	LOST	OK	OK	OK	OK	DUP

OK = 协议功能正确
DUP = 协议产生了一个重复消息
LOST = 协议丢失了一个消息

图 6-18 客户和服务器的策略组合

进一步精心修改协议也无济于事。即使在服务器试图写之前，客户与服务器再多交换几段，使得客户确切地知道将要发生什么事情，客户也没有办法知道崩溃动作发生在写操作之前还是之后。结论是不可避免的：在事件不同步的基本规则下——即单独的事件一个接着一个发生，而不是同时发生，那么主机的崩溃和恢复就无法做到对上层透明。

按照更加一般的术语表达，这个结论可以重新叙述为：“从第 N 层崩溃中的恢复工作只能由第 $N+1$ 层完成”，并且仅当高层保留了问题发生前的足够状态信息时才有可能恢复。这与上面提到的观点一致，只要连接两端记录了它当前的状态信息，传输层才能够从网络层的故障中恢复过来。

这个问题促使我们思考究竟所谓的端到端确认意味着什么。原则上，传输协议是端到端的，而不像下面层次那样是链式的。现在请考虑这样的情形：一个用户向一个远程数据库发出了一些事务请求。假设远程传输实体先将段传递给上一层，然后再确认。在这个例子中，即使用户机器收到返回的确认也不一定意味着远程主机一定能够撑足够长的时间来真正完成数据库的更新操作。一个真正的端到端确认或许没有可能做得到，因为一旦接收到这个确认就意味着工作确实已经完成，而缺乏确认则表明工作尚未完成。（Saltzer 等，1984）详细地讨论了这一点。

6.3 拥塞控制

如果许多机器上的传输实体以太快的速度发送太多的数据包，就会使得网络不堪重负而变得拥塞，继而数据包被延迟和丢失，从而导致网络性能严重下降。避免这个问题的拥塞控制是网络层和传输层的共同责任。拥塞发生在路由器上，因此在网络层检测拥塞。然而，拥塞究竟还是由传输层注入到网络中的流量引起的，因此控制拥塞的唯一途径是传输层放缓往网络中发送数据包的速度。

在第 5 章，我们学习了网络层的拥塞控制机制。本节，我们将学习问题的另一半，即传输层的拥塞控制机制。在描述完拥塞控制的目标后，我们将描述主机如何调节它们往网络发送数据包的速率。Internet 的拥塞控制严重依赖于传输层，特殊的算法被嵌入到 TCP 和其他协议中。

6.3.1 理想的带宽分配

在我们描述如何调节流量之前，我们必须理解运行拥塞控制算法要努力达到的目标是什么。也就是说，我们必须说明所谓一个好的拥塞控制算法在网络中的运行状态。拥塞控制算法的目标是更加易于避免拥塞，即为使用网络的传输层找到一种好的带宽分配方法。一个好的带宽分配方法能带来良好的性能，因为它能利用所有的可用带宽却能避免拥塞，而且它对于整个竞争的传输实体是公平的，并能快速跟踪流量需求的变化。我们将依次精确地说明这些标准。

效率和功率

为整个传输实体有效分配带宽应该利用所有可用的网络容量。然而，假设存在一条

100 Mbps 的链路，5 个传输实体共同使用这条链路，每个实体将获得 20 Mbps，如果真这样想就大错特错了。要想获得良好的性能，它们获得的带宽应该小于 20 Mbps。其中的缘由于流量通常呈现突发性。回忆一下，在 5.3 节我们描述过实际吞吐量（或到达接收端的有用数据包速率）是提交负载的函数。图 6-19 给出了这条曲线以及延迟曲线，与正常输出一样，延迟也是提交负载的函数。

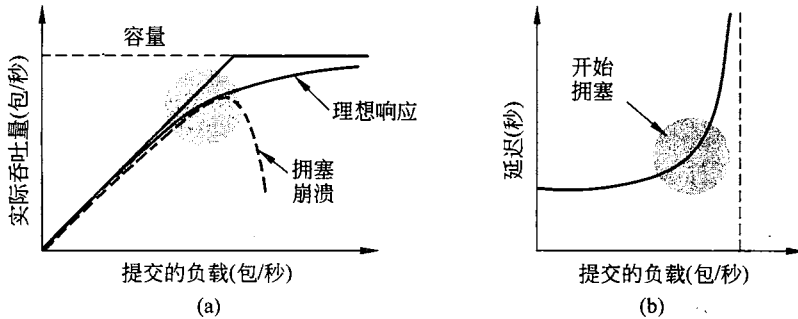


图 6-19
(a) 实际吞吐量；(b) 延迟作为提交负载的函数

在图 6-19 (a) 中，随着负载的增加实际吞吐量最初以同样的速度增加，但随着负载接近网络容量，实际吞吐量的上升逐渐增多。由于突发流量可能导致网络内缓冲区偶尔被充满并造成一些数据包丢失，因而出现实际吞吐量的衰减。如果传输协议设计不当，重传的数据包依然会被延迟但还未丢弃，此时网络内的数据包越积越多最终拥塞崩溃。在这种状态下，发送端拼命地发送数据包，但完成的有益工作只能增加一点点。

图 6-19 (b) 给出了相应的延迟变化情况。最初的延迟是固定的，表示穿过整个网络的传播延迟。随着负载接近网络容量，延迟逐步上升，开始上升速度比较缓慢，然后骤然上升。这也是因为突发流量在高负荷下被堆积起来的缘故。延迟实际上不可能真正达到无穷，除非在一个路由器有无限大缓冲区的模型中。相反，数据包在经历了最大的缓冲延迟后被路由器丢弃。

对于实际吞吐量和延迟，在拥塞出现时性能开始下降。直观地说，如果我们逐步加大分配的带宽，将从网络获取最佳的性能，直到延迟开始迅速攀升的那点。而这一点恰好低于网络容量。为了标识它，(Kleinrock, 1979) 提出了功率的度量，其中

$$\text{功率} = \text{负载} / \text{延迟}$$

功率最初将随着提交负载的上升而上升，延迟仍然很小并且基本保持不变；但随着延迟快速增长功率将达到最大，然后开始下降。达到最大功率的负载表示了传输实体放置在网络上的有效负载。

最大-最小公平性

在前面的讨论中，我们没有涉及如何在多个传输实体之间划分带宽。这看上去只回答了一个简单问题——给全部的发送端以均等的带宽比例，但这里有几点值得考虑。

也许，首先考虑的是拥塞控制究竟要解决什么问题。毕竟，如果网络给发送端一些带宽供它使用，那么发送端应该只使用那么多的带宽。然而，通常情况下，网络无法为每个流或者连接执行严格的带宽预留。如果网络支持服务质量，那么它们将为某些流预留带宽，

但是许多连接将寻求可用的任何带宽，或者被网络合并在一起共同分配带宽。例如，IETF 的区分服务就将流量分成两类，每个类中的连接竞争带宽的使用。IP 路由器通常让所有的连接竞争相同的带宽。在这种情况下，正是拥塞控制机制来为竞争的各个连接分配带宽。

第二个要考虑的是公平对网络中的流意味着什么。其实这点很简单，如果 N 个流使用一条链路，在这种情况下，它们都应该拥有 $1/N$ 的带宽（虽然在流量是突发性的时候，效率决定了它们能使用的要略微少一些）。但如果有不同的流，并且这些流的网络路径出现重叠，会发生什么情况？例如，一个流可能穿越三条链路，其他流可能跨越一条链路。走三条链路的流将消耗更多的网络资源。给它分配的带宽少于只走一条链路的流获得的带宽可能在某种意义上更公平。当然有可能通过减少三条链路流的带宽来支持更多的一条链路的流。这一点体现了公平与效率之间的内在张力。

不过，我们将采取的公平概念不依赖于网络路径的长度。即使这个简单的模型，让连接分配同等比例的带宽也有点复杂，因为不同的连接将采用通过网络的不同路径，这些路径本身有不同的容量。在这种情况下，有可能一个流在下行链路上被阻塞，并且在上行链路上获得比其他流更小的流量；此时减少其他流的带宽不仅会使得这些流慢下来，而且对瓶颈流也不会有任何帮助。

公平的形式是最大-最小公平（max-min fairness），通常表示理想的网络使用情况。最大-最小公平分配指的是，如果分配给一个流的带宽在不减少分配给另一个流带宽的前提下无法得到进一步增长，那么就不给这个流更多带宽。也就是说，增加一个流的带宽只会让不太富裕的那些流的情况变得更糟。

让我们看一个例子。图 6-20 显示了最大最小公平分配方法，在这个网络中存在 4 个流，分别标识为 A、B、C 和 D。路由器之间的每条链路具有相同的容量，用 1 个单位表示，虽然在一般情况下链路的容量是不同的。3 个流竞争左下角位于路由器 R4 和 R5 之间的链路。因此，这些流中的每一个都得到 $1/3$ 的链路容量。其余的流，A 与 B 竞争从路由器 R2 到 R3 的链路。由于 B 已经分配得到 $1/3$ ，A 得到剩余的 $2/3$ 链路容量。请注意，所有的其他链路都有空余容量。但是，这种空余的容量不能给予任何其他流使用，除非降低另一个流的更低容量。例如，如果把路由器 R2 和 R3 之间的带宽给 B 分配得多一些，那必定要降低流 A 的容量。这对于已经有更多带宽的流 A 来说是合理的。然而，流 C 或 D（或两者）的容量必须减少才能给流 B 更多的带宽，而这些流的带宽将小于 B。因此，这种分配方法是最大-最小公平的。

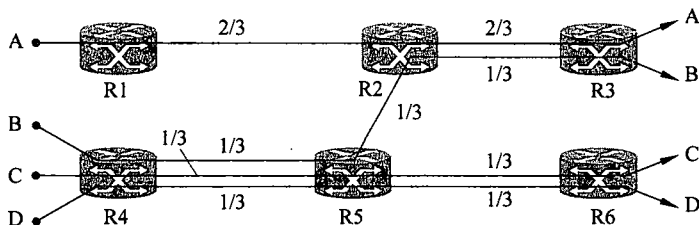


图 6-20 针对 4 个流的最大-最小带宽分配

只要获知网络的全局知识最大-最小分配就可以计算得出来。一种直观的思考方式是想象所有的流从速率零开始，然后缓慢地增加速率。当任何一个流的速率遇到瓶颈，就停止

该流的速率增加；所有其他的流继续增加各自的速率，平等共享可用容量，直到它们也达到各自的“瓶颈”。

第三个需要考虑的因素是在什么程度上考虑公平性。一个网络的公平性可以体现在连接上、每一对主机之间的所有连接上，或者每个主机的所有连接上。我们在 5.4 节讨论加权公平队列（WFQ, Weighted Fair Queueing）时考查过这个问题，并得出了这样一个结论，即每种定义方法都存在着自己的问题。例如，如果针对每个主机定义公平性，则意味着一个繁忙的服务器将不会比手机获得的带宽更多；而若对每个连接定义公平性，实际上又起到鼓励主机打开更多连接的效果。鉴于没有明确的答案，经常把公平性视为针对每个连接，但确切的公平通常不予考虑。实际上，连接得不到带宽远比所有的连接得到精确的相同数量的带宽要重要得多。事实上，有可能用 TCP 打开多个连接，并且更积极地参与竞争带宽。这种战术通常被用于那些需要高带宽的应用，比如对等文件共享系统 BitTorrent。

收敛

最后一个标准是拥塞控制算法能否快速收敛到公平而有效的带宽分配上。上面讨论的理想操作点假设了一个静态的网络环境。然而，网络中的连接总是来来去去，而且一个给定连接所需要的带宽也会随时间而变化，例如，一个用户浏览网页和偶尔会下载大的视频。

由于需求的变化，网络的理想操作点也随着时间推移而改变着。一个好的拥塞控制算法，应迅速收敛到理想的操作点，并跟踪这随时变化的操作点。如果收敛速度太慢，则算法永远无法接近已经改变的操作点；如果算法不稳定，它也可能在某些情况下无法收敛到正确的操作点，或者甚至围绕着正确的操作点振荡。

图 6-21 给出了一个带宽分配的例子，它能随着时间变化并且快速收敛。最初，流 1 拥有全部的带宽。一秒钟以后，流 2 启动，它也需要带宽。带宽分配算法迅速改变分配结果，给每个流总带宽一半的份额。在第 4 秒，第三个流加入。然而，这个流仅仅使用了 20% 的带宽，这低于它应该获得的公平份额（应该是三分之一）。流 1 和流 2 迅速调整，将可用带宽每个调整到 40%。在第 9 秒，第二个流离开，第三个流保持不变。第一个流迅速捕获 80% 的带宽。在整个过程中的任何时候，分配的总带宽大约是 100%，从而网络得到充分利用，各个竞争的流得到平等的待遇（但不必使用比它们需要的更多带宽）。

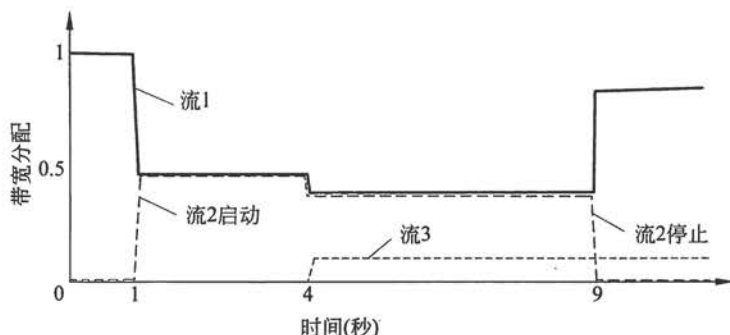


图 6-21 随着时间的推移而变化的带宽分配

6.3.2 调整发送速率

现在是时候进入主要的课程学习了。我们如何调整发送速率，以便获得一个理想的带

宽分配？发送速率可能受到两个方面因素的限制。首先是流量控制，在接收端没有足够缓冲区的情况下，必须进行流量控制。第二个因素是拥塞控制，在网络容量不足的情况下必须继续进行拥塞控制。在图 6-22 中，我们可以看到用液压说明的这个问题。在 6-22 (a) 中，一根粗粗的管道接到一个小容量的接收器。这是流量控制限制的情况。只要发送端不发送比漏桶能容纳的更多水，水就不会因溢出而丢失。在图 6-22 (b) 中，限制因素不是桶的容量，而是网络内部的承载能力。如果太多的水太快地注入，它就会回流，因而造成丢失（在这种情况下，水从漏斗溢出）。

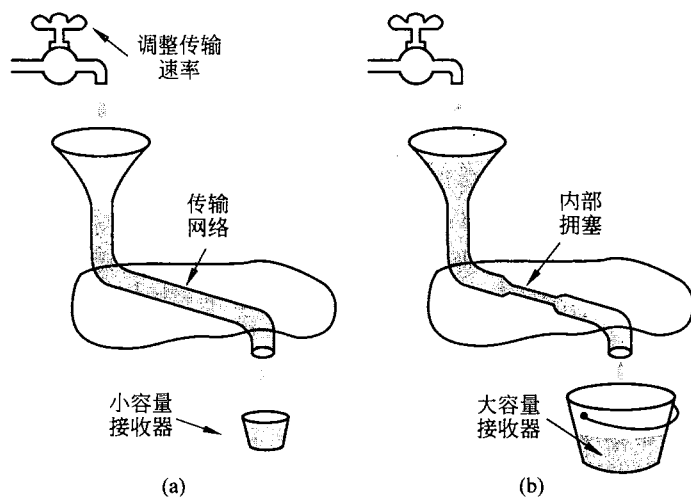


图 6-22

(a) 一个快速网络往小容量接收器中注水；(b) 一个小容量网络往大容量接收器中注水

类似的这种情况可能会出现在发送端，由于传输速度过快导致数据包丢失。然而，它们有不同的原因，因而需要不同的解决方案。我们已经讨论过采用一个可变大小窗口的流量控制解决方案。现在，我们考虑拥塞控制解决方案。由于上述这些问题都可能发生，所以传输协议一般需要同时运行两种解决方案，并且当其中一个问题发生时放慢发送速度。

传输协议调节发送速率的方式依赖于网络返回的反馈意见形式。不同的网络层可能会返回不同类型的反馈。反馈可能是显式的，也可能是隐式的；而且反馈信息可能精确，也可能不精确。

一个显式并精确的设计例子是路由器告诉数据包的源端，它们能以多大的速率发送数据包。在文献资料中，**显式拥塞协议**（XCP, eXplicit Congestion Protocol）就是以这种方式工作的（Katabi 等，2002）。另一个显式的但不精确的设计例子是使用 TCP 的**显式拥塞通知**（ECN, Explicit Congestion Notification）。在这个设计方案中，路由器在经历拥塞的数据包中设置警告比特来警告发送端放慢速率，但它们不告诉源端该减缓多少。

在其他设计方案中，没有明确的信号。FAST TCP 测量端到端往返延迟，并用该指标作为避免拥塞的信号（Wei 等，2006）。最后，在今天的 Internet 上最流行的拥塞控制形式是带有尾丢包的 TCP 或 RED 路由器，数据包的丢失要通过推断得出，并且丢包被当作网络是否变得拥挤的信号。这种形式的 TCP 有许多变种，包括 Linux 系统下采用的 CUBIC TCP（Ha 等，2008）。组合信号也是有可能的。例如，Windows 的 Compound TCP 同时使用了丢

包和延迟作为拥塞的反馈信号（Tan 等，2006）。图 6-23 给出了这些设计思想的概况。

协议	信号	显式否	精确否
XCP	使用速率	是	是
TCP	拥塞警告	是	否
FAST TCP	端到端延迟	否	是
Compound TCP	数据包丢失&端到端延迟	否	是
CUBIC TCP	数据包丢失	否	否
TCP	数据包丢失	否	否

图 6-23 一些拥塞控制机制的信号

如果给出一个显式的和精确的信号，那么传输实体就可以利用该信号来调整发送速率到新的操作点。例如，如果 XCP 告诉发送端使用多大的发送速率，那么发送端只需简单地采用该速率来发送数据包。然而，在其他情况下，涉及一些猜测工作。在没有拥塞信号的情况下，发送端应该增加发送速率；当给出拥塞信号时，发送端应降低发送速率。增加或减少速率的方式由控制法则（control law）决定。这些法则对性能有重大的影响。

（Chiu 和 Jain，1989）研究了二进制拥塞反馈的情况，他们得出的结论是加法递增乘法递减（AIMD, Additive Increase Multiplicative Decrease）法则达到有效和公平操作点的适当流量规则。为了表明这种情况，他们构建了一个简单实例的图形表示，在这里有两个流竞争单条链路的带宽。在图 6-24 中，分配给用户 1 的带宽用 x 轴表示，分配给用户 2 的带宽用 y 轴表示。当分配满足公平性时，两个用户将获得相同数量的带宽。这在图中用一条虚的公平线表示。当分配的带宽总和为 100% 时，即达到链路容量时分配是有效的，这在图中用一条虚线效率线表示。当两个用户分配的带宽总和超过这条线时，网络就给两个用户发出一个拥塞信号。这些线的交点就是理想的操作点，此时两个用户具有相同的带宽，并且所有的网络带宽都被使用上。

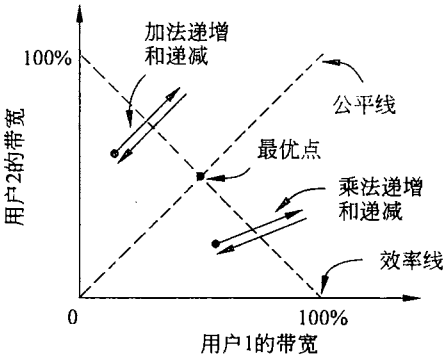


图 6-24 加法递增乘法递减的带宽调整

如果用户 1 和用户 2 都随着时间推移按加法递增法则增加各自的带宽，我们考虑从某时候开始带宽的分配会出现什么情况。例如，两个用户以每秒增加 1 Mbps 的速度递增各自的发送速率。最终，操作点穿过效率线，两个用户都收到一个从网络传来的拥塞信号。在这个阶段，他们必须减少各自分配的带宽。然而，加法递减只会导致他们沿着递增线振荡。这种情况如图 6-24 所示。这种行为将保持操作点接近效率线，但它并不一定是公平的。

同样，考虑当两个用户随着时间的推移按乘法（加倍）法则递增带宽直到他们收到一个拥塞信号的情况。例如，用户可能每秒增加 10% 的发送速率。然后，如果他们乘法递减各自的发送速率，用户的操作点将沿着乘法线振荡。这种行为也如图 6-24 所示。乘法线的斜率和加法线的斜率不同（它指向源点，而加法线呈 45° 角）。不过它也没有更好的选择。在这两种情况下，用户都难以收敛到同时兼顾公平与效率的最佳发送速率。

现在考虑这样的情况，用户加法递增他们的带宽，然后当收到拥塞信号时乘法递减他

们的带宽。这种行为就是 AIMD 控制法则，如图 6-25 所示。由此可以看出，这种行为的轨迹路径能收敛到兼顾公平与效率的最佳点。这种收敛不管从什么出发点开始都能发生，因而使得 AIMD 得到了广泛使用。同样的论点，唯一的其他组合，即乘法递增和加法递减，将会偏离最佳点。

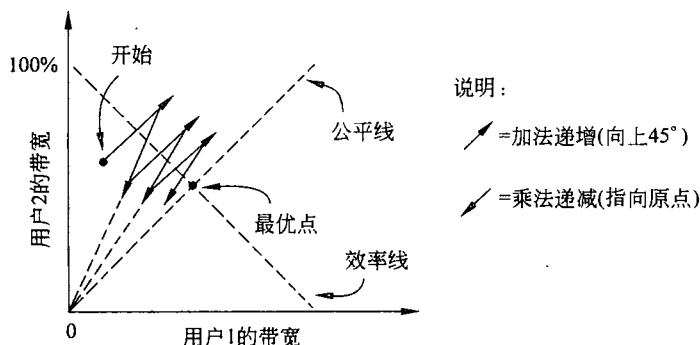


图 6-25 加法递增乘法递减 (AIMD) 控制法则

AIMD 是 TCP 采用的拥塞控制法则，它基于这个观点和另一个稳定性观点（即驱使网络拥堵非常容易而从中恢复却很难，所以递增政策应轻柔，而递减政策应积极）。但这个法则不是那么的公平，因为 TCP 连接根据每次的往返时间测量值来调整窗口的大小，而不同的连接有不同的往返时间。这导致在所有其他条件都相同的情况下，接近主机的连接比远离主机的连接获得的带宽更多。

在 6.5 节，我们将详细描述 TCP 是如何实现 AIMD 控制法则来调整发送速率并实行拥塞控制的。这个任务做起来比听起来要困难得多，因为要在某个时间间隔测量速率，而且流量是突发的。实际上通常使用的策略不是直接调整发送速率，而是调整滑动窗口的大小。TCP 就是使用了这种策略。如果窗口大小是 W ，往返时间是 RTT ，则等价的发送速率是 W/RTT 。这种策略很容易和流量控制机制结合起来，因为流量控制机制已经使用了一个窗口。同时这种策略还有个优点，发送端可以通过确认来测定数据包的速度，而且如果它停止接收数据包离开网络的报告那么还可以放缓 RTT 。

最后一个问题是关于多协议的竞争。网络中可能存在许多不同的传输协议，都往网络发送流量。为了避免拥塞，不同的协议采用了不同的控制法则，如果这些不同的协议竞争时会发生什么？那就是带宽分配不平等。由于 TCP 是 Internet 拥塞控制的主要形式，因此设计新的传输协议要承受相当显著的社会压力，新协议必须能与 TCP 进行公平的竞争。早期的流媒体协议就是因为产生了不公平竞争问题，结果是过度减少了 TCP 吞吐量。这种现象导致了 TCP 友好 (TCP-friendly) 拥塞控制这一概念的提出。在 TCP 友好的拥塞控制中，TCP 和非 TCP 传输协议可以自由地混合，而没有任何不良影响 (Floyd 等，2000)。

6.3.3 无线问题

诸如 TCP 这样的传输协议实现拥塞控制应该独立于下面的网络层和链路层技术。理论上这是好的，但在实际的无线网络中却存在问题。主要的问题在于通常的传输协议一般把丢包视作拥塞发生的信号，包括我们刚刚讨论的 TCP 也是这样认为的。而在无线网络中，

丢包几乎都是因为传输错误引起的。

采用 AIMD 控制法则，较高的吞吐量要求非常小的数据包丢失。（Padhye 等，1998）分析表明吞吐量作为丢包率的逆平方根上升。在实践中这意味着快速 TCP 连接的丢失率非常小；1%是中等程度的丢失率，待丢失率达到 10%时连接就几乎停止工作了。然而，无线网络，比如 802.11 局域网，至少 10%的帧丢失率是很常见的情况。这种差异意味着如果没有保护测量措施，把丢包率作为一个信号的拥塞控制方案将把运行在无线链路上的连接压制到非常低的速率，而这显然是没有必要的。

为了在无线网络中也能正常工作，拥塞控制算法观察到的丢包只能是那些因带宽不足造成的丢包，而不能把由于传输错误造成的丢包也算在内。这个问题的一种解决方案是通过使用无线链路上的重传机制把无线链路的丢包掩盖起来。例如，802.11 使用停-等式协议来传递每个帧，在向上层报告丢包之前，如果需要会进行多次重传。在正常情况下，每个数据包的交付对高层是不可见的，尽管可能曾经发生过瞬间的传输错误。

图 6-26 显示了一条由有线和无线链路组成的路径，传输层采用了屏蔽策略。有两个方面需要注意。首先，发送端并不一定要知道该路径包括一条无线链路，因为它所能看到的只是与它连接的有线链路。Internet 上的路径是异构的，而且不存在一种通用的方法来告知发送端路径由什么链路组成。这样的现状加深了拥塞控制问题的复杂性，因为没有简单的方法使得无线链路采用一个协议而有线链路采用另一个协议。

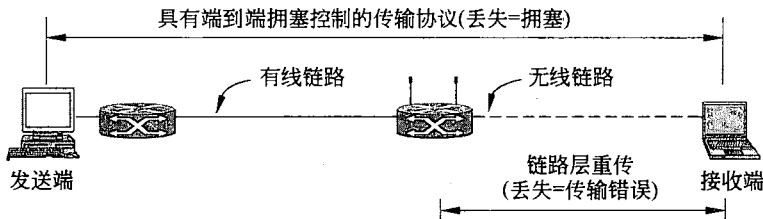


图 6-26 针对具有无线链路的路径的拥塞控制

第二个方面是一道谜。该图显示了由丢失驱动的两个机制：链路层的帧重传和传输层的拥塞控制。让人困惑的是这两种机制如何并存而不会混淆。毕竟，丢失应该只引起一个机制采取行动，因为它要么是一次传输错误，要么是一个拥塞信号。它不能同时是这两个原因造成的。如果两种机制都采取行动（重传帧并且减慢发送速率），那么我们又回到了原先的传输问题，即在无线链路上的运行过于缓慢。考虑一下这个难题，看看你是否能找到这个问题的解决方案。

解决的办法是这两种机制作用在不同的时间尺度。对于诸如 802.11 的无线链路，链路层的重传发生在微秒到毫秒级；而传输层的丢失计时器发生在毫秒到秒。区别就在于这三个时间尺度。这样的话，在传输层推测出丢包之前，无线链路已经检测出丢帧并重传帧，由此修复了传输错误。

屏蔽策略足以让大多数传输协议在大多数无线链路上运行得良好。但是，这并不总是一个合适的解决方案。某些无线链路的往返时间很长，比如卫星。对于这些链路，必须使用其他技术来掩盖丢包事实，比如前向纠错（FEC, Forward Error Correction），或者传输协议在拥塞控制时必须使用一种非丢失（non-loss）信号。

与无线链路上的拥塞控制有关的第二个问题是链路的容量可变。也就是说，随着时间的推移无线链路的容量会发生变化，而且因为节点的移动或者信道条件的变化而引起的信噪比的变化，链路容量的变化有时非常突然。这点与有线链路有很大的不同，有线链路的容量是固定不变的。传输协议必须适应无线链路容量的变化，否则将阻塞网络，或者无法使用可用的链路容量。

这个问题的一个可能解决办法是不解决，也就是说根本不必担心这个问题。这种策略其实是可行的，因为拥塞控制算法早就应该能处理新用户进入网络或现有用户改变发送速率等这些情况。即使有线链路的容量是固定的，其他用户不断变化的行为本身也会对给定用户的可用带宽带来变化。因此，在一条具有 802.11 无线链路的路径上简单地运行 TCP，有可能获得合理的性能。

然而，当存在太多的无线变异时，继续使用专为有线链路设计的传输协议可能会有麻烦，并且由此带来的性能比较差。在这种情况下，唯一的解决方案是专门设计一个针对无线链路的传输协议。一个特别具有挑战性的环境是无线网状网。在这样的网络中，多个相互干扰的无线链路彼此交错，路由因移动性而不断地发生着变化，并且存在大量的丢包。这方面的研究正在进行之中。（Li 等，2009）给出了一个无线传输协议设计的例子。

6.4 Internet 传输协议：UDP

Internet 的传输层有两个主要协议，无连接和面向连接各一个，两个协议互为补充。无连接协议是 UDP，它除了给应用程序提供发送数据包功能并允许它们在所需的层次之上架构自己的协议之外，几乎没有做什么特别的事情。面向连接的协议是 TCP，该协议几乎做了所有的事情。它建立连接，并通过重传机制增加了可靠性，同时还进行流量控制和拥塞控制，代表使用它的应用程序做了所有的一切。

在下面的章节中，我们将学习 UDP 和 TCP。我们从 UDP 开始，因为它是最简单的传输协议。我们也会考察 UDP 的两种应用模式。由于 UDP 是传输层协议，通常运行在操作系统中，而使用 UDP 的协议通常运行在用户空间，因而可以把对 UDP 的使用视为应用程序。然而，它们的使用技术对许多应用都有用，而且把这些技术视作属于传输层更好，所以这里我们将介绍它们。

6.4.1 UDP 概述

Internet 协议集支持一个无连接的传输协议，该协议称为用户数据报协议（UDP，User Datagram Protocol）。UDP 为应用程序提供了一种无需建立连接就可发送封装的 IP 数据报的方法。RFC 768 描述了 UDP。

UDP 传输的段（segment）由 8 字节的头和有效载荷字段构成。图 6-27 描述了头信息。两个端口（port）分别用来标识源机器和目标机器内部的端点。当一个 UDP 数据包到来时，它的有效载荷被递交给与目标端口相关联的那个进程。在调用了 BIND 原语或者其他某种类似的原语之后，这种关联关系就建立了起来，正如我们在图 6-6 中介绍 TCP 时已经看过

的那样（UDP 的绑定过程是一样的）。可以把端口看作应用程序租来接收数据包的邮箱。在我们描述同样使用端口的 TCP 时，还将对此多说几句。实际上，采用 UDP 而不是原始 IP 的最主要价值在于增加了源端口和目标端口。如果没有端口字段，传输层将无从知道如何处理每个入境数据包；而有了端口字段之后，它就能把内嵌的段递交给正确的应用程序处理。

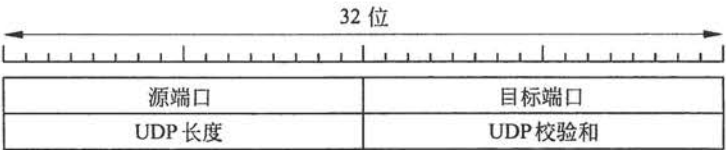


图 6-27 UDP 头格式

当接收端必须将一个应答返回给源端时，源端口（Source Port）字段则是必需的。只要将入境段中的源端口字段复制到出境段中的目标端口（Destination Port）字段，接收端就指定了由发送端机器上的哪个进程来接收应答。

UDP 长度（UDP Length）字段包含 8 字节的头和数据两部分的总长度。最小长度是 8 个字节，刚好覆盖 UDP 头。最大长度是 65 515 字节，恰好低于填满 16 比特的最大字节数，而这是由 IP 数据包限制的。

一个可选的校验和（UDP Checksum）字段还提供了额外的可靠性。它校验头、数据和一个概念性的 IP 伪头。执行校验和计算时，校验和字段先被设置为零，如果数据字段的长度是奇数则用零填充成偶字节。校验和算法很简单，先按 16 位字的补码相加求和，然后再取总和的补码。因此，当接收端对整个段计算校验和时，要包括 UDP 校验和字段，正确的结果应该为 0。如果发送端没有计算校验和，则将该字段值填为 0，因为补码计算结果可能碰巧真的是 0，则存储为全 1。然而，关闭校验和计算不是明智之举，除非数据传输的质量并不重要（例如，数字化语音）。

IPv4 的伪头部如图 6-28 所示，它包含源机器和目标机器的 32 位 IP 地址、UDP 的协议号（17），以及 UDP 段（包括头）的字节计数。IPv6 的伪头与其类似，但有些许不同。在 UDP 校验和计算中包含伪头将有助于检测出被错误递交的数据包，但是传输层在计算校验和时需要地址的做法违反了协议分层原则，因为 IP 地址属于 IP 层，而不属于 UDP 层。TCP 在计算校验和的时候也使用了同样的伪头。

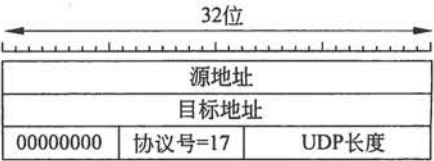


图 6-28 UDP 校验和包括了 IPv4 伪头

或许值得明确一提的是 UDP 没有做的事情。它没有流量控制、拥塞控制，或者接收到一个坏段后的重传机制。所有这一切都必须由用户进程来完成。它只是提供了一个与 IP 协议的接口，并在此接口上增加通过端口号复用多个进程的功能，以及可选的端到端错误检测功能。这就是 UDP 所做的一切。

对于需要对数据包流实施精确控制、错误检测或者时序功能的应用，UDP 提供的服务恰到好处。UDP 协议特别有用的一个领域是客户机-服务器应用开发。一般情况下，客户端向服务器发送一个简短的请求报文，并期待来自服务器的简短回复报文。如果请求或回复报文丢失，客户端就会超时，然后再试一次。以 UDP 实现的代码相比需要初始建立连接的协议（比

如 TCP) 不仅代码简单, 而且需要交换的报文也少 (对于连接来说, 每个方向都需要一份)。

以这种方式使用 UDP 的一个应用是域名系统 (DNS, Domain Name System), 我们将在第 7 章中学习这个系统。简单地说, 如果一个程序需要查询某个主机名的 IP 地址, 比如 `www.cs.berkeley.edu`, 那么它可以给 DNS 服务器发送一个包含该主机名的 UDP 数据包。服务器用一个包含了该主机 IP 地址的 UDP 数据包作为应答。事先不需要建立连接, 事后也不需要释放连接。只有两条消息通过网络就够了。

6.4.2 远程过程调用

在某种意义上, 向一台远程主机发送一个消息并获得一个应答, 就如同在编程语言中执行一个函数调用一样。在这两种情形下, 启动时都需要提供一个或者多个参数, 然后获得一个结果。这种观察导致人们试图将网络上的请求-应答交互过程安排成像过程调用那样。这样的安排使得网络应用更加易于编程也更为人们所熟悉。例如, 请想象一个名为 `get_IP_address(host_name)` 的过程, 它的工作方式是向 DNS 服务器发送一个 UDP 数据包, 然后等待应答; 如果应答返回的速度不够快, 则超时并重试。通过这种方式, 向程序员隐藏了网络的所有细节。

这个领域中的关键工作是由 (Birrell 和 Nelson, 1984) 完成的。一言以蔽之, Birrell 和 Nelson 的建议是允许本地程序调用远程主机上的过程。当机器 1 上的进程调用机器 2 上的一个过程时, 机器 1 上的调用进程被挂起, 而机器 2 上的被调用过程则开始执行。信息以参数的形式从调用方传输到被调用方, 而过程的执行结果则从反方向传递回来。应用程序看不到任何消息的传递。这项技术称为**远程过程调用 (RPC, Remote Procedure Call)**, 目前已经成为许多网络应用的基础。传统上, 调用过程称为**客户**, 被调用过程称为**服务器**, 这里我们将沿用这些名字。

RPC 背后的思想是尽可能地使一个远程过程调用看起来像本地过程调用一样。在最简单的形式中, 为了调用一个远程过程, 客户程序必须绑定 (链接) 到一个小的库过程, 这个库过程称为**客户存根 (client stub)**, 它代表了客户地址空间中的服务器过程。类似地, 服务器需要绑定到一个称为**服务器存根 (server stub)** 的过程。正是这些过程, 隐藏了从客户机不在本地调用服务器的事实。

图 6-29 给出了执行 RPC 的实际步骤。第 1 步是客户调用客户存根。这是一个本地过程调用, 其参数按照常规的方式压入到栈中。在第 2 步, 客户存根将参数封装到一个消息中, 然后通过系统调用发送该消息。封装参数的过程称为**列集 (marshalling)**。在第 3 步, 操作系统将消息从客户机器发送到服务器机器上。在第 4 步, 操作系统将入境数据包传递给服务器存根。最后, 在第 5 步中, 服务器存根利用**散集 (unmarshaled)** 得到的参数调用服务器过程。调用的结果沿着相反的方向按同样的路径传递。

这里需要注意的关键点是用户编写的客户过程只是按照普通过程调用的方式来调用客户存根, 而且客户存根与服务器过程有同样的名字。由于客户过程和客户存根在同样的地址空间中, 所以, 参数的传递也是按通常的方式进行。类似地, 服务器过程被同一地址空间中的一个过程调用, 并且它接收到的也正好是它所期望的参数。按照这种方式, 网络通信并不是在套接字上完成输入和输出, 而是通过仿造一个普通的过程调用来完成的。

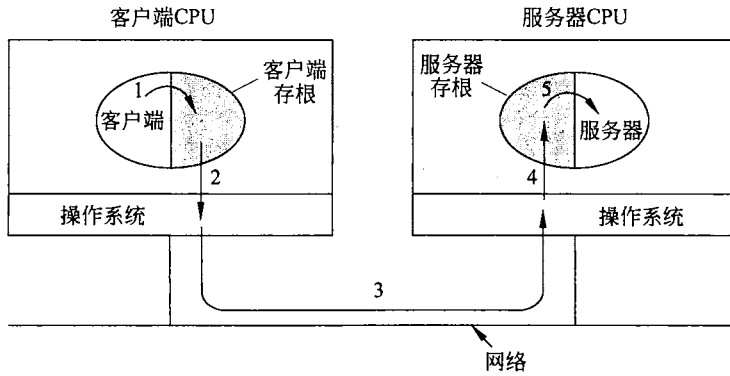


图 6-29 远程过程调用的步骤（阴影部分表示存根）

不管 RPC 的概念有多么的精巧，这其中还隐藏着一些陷阱。最大的陷阱是指针参数的用法。在正常情况下，将指针传递给一个过程不是问题。被调过程可以像调用方那样使用这个指针，因为双方生存在同样的虚拟地址空间中。而对于 RPC，传递指针是不可能的，因为客户和服务器位于不同的地址空间中。

在有些情况下，可以使用一些技巧来实现指针的传递。假设第一个参数是一个指向某个整数 k 的指针。客户存根可以对 k 进行列集，并发送给服务器；然后，服务器存根创建一个指向 k 的指针，并且将该指针传递给服务器过程，而这正是服务器过程所期望的。当服务器过程将控制权返回给服务器存根时，后者将 k 送回给客户。在客户端，新的 k 值被复制到老的 k 中，就好像服务器对它做了修改一样。实际上，整个过程相当于用按复制-恢复调用（call-by-copy-restore）代替了按引用调用（call-by-reference）的标准调用序列。不幸的是，这种技巧并不总是能正确地工作，例如，若指针指向一个图形或者其他复杂的数据结构。由于这个原因，对于远程调用的参数必须强加一些限制。

第二个问题是在一些弱类型的语言中（比如 C），编写一个计算两个矢量（即数组）内积，但不指定矢量大小的过程是完全合法的。每个矢量都有一个专门的值作为终止标记，而且该终止值只有调用过程和被调用过程才知道。在这种情况下，客户存根要想对参数执行列集操作从本质上讲是不可能的：它无从知道该如何确定参数的长度。

第三个问题在于并不是总能推断出参数的类型，即使从正式的规范或者代码本身都未必能做得到。一个例子是 `printf`，它可能有任意多个参数（至少一个），而且这些参数可以是整数、短整数、长整数、字符、字符串、可变长度的浮点数和其他类型的任意组合。试图以远程过程的方式来调用 `printf` 不太现实，因为 C 语言对类型的要求特别宽松。然而，有一条规则是这样的：只要不用 C（或者 C++）编写程序，那么就可以使用 RPC，但这种说法没有得到程序员的广泛认可。

第四个问题涉及全局变量的使用。正常情况下，调用过程和被调用过程除了通过参数进行通信以外，也可以使用全局变量作为通信的手段。如果被调用的过程被转移到了远程机器上，那么，这样的代码就会失败，因为全局变量无法再为双方所共享了。

这些问题并不意味着 RPC 就没有希望。事实上，它的应用仍然十分广泛，但是在实际运用中，需要有一些限制才能保证它正常工作。

关于传输层协议，UDP 是一个实现 RPC 的良好基础。请求和应答可作为单个 UDP 数

据包以最简单的形式发送，并且可以快速进行操作。然而，RPC 的实现必须包括其他机器。因为请求或应答数据包可能会丢失，客户端必须维持一个计时器用来重发请求。请注意，应答数据包可当作一个请求的隐含确认，因此该请求不再需要单独确认。某些时候，参数或结果可能大于最大的 UDP 数据包大小，在这种情况下需要传递大块消息的某种协议。如果多个请求和应答可以重叠（在并发编程情况下），则需要一个标识符把应答和请求相匹配。

一个更高层次的关注是操作可能不是幂等的（即重复安全）。一个简单的幂等操作例子是 DNS 请求和应答。客户如果没有收到答复，可以放心地一次又一次地重发请求，无须担心服务器是否永远收不到请求，或者返回的应答报文会丢失。当应答报文终于来临时，答案将是相同的（假设在此期间 DNS 数据库没有更新）。然而，并非所有的操作都是幂等的，因为它们有严重的副作用，比如递增一个计数器。对于这些非幂等操作 RPC 需要较强的语义，以便当程序员调用一个过程时它不会被多次执行。在这种情况下，可能有必要建立一个 TCP 连接并在连接上发送请求，而不是使用 UDP 发送请求报文。

6.4.3 实时传输协议

客户-服务器 RPC 是 UDP 被广泛应用的一个领域，UDP 的另一个应用领域是实时多媒体应用。尤其是，随着 Internet 广播电台、Internet 电话、音乐点播、视频会议、视频点播和其他的多媒体应用越来越普及，人们发现每一种应用都在或多或少地重复设计几乎相同的实时传输协议。逐渐地人们意识到，为多媒体应用制定一个通用的实时传输协议是一种很好的想法。

于是实时传输协议（RTP，Real-time Transport Protocol）诞生了。RTP 由 RFC 3550 描述，目前已经广泛应用于多媒体应用程序。我们将描述实时传输的两个方面。第一个是 RTP 协议，它以数据包形式传输音频和视频数据。第二个主要是接收端的处理，涉及在正确的时间播放音频和视频。这些功能在协议栈中的位置如图 6-30 所示。

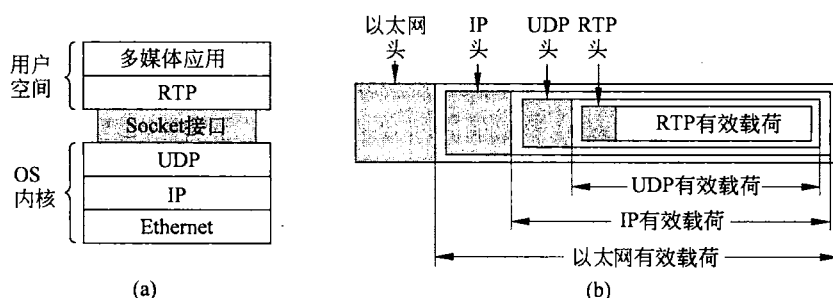


图 6-30
(a) RTP 在协议栈中的位置；(b) 数据包的嵌套

RTP 通常运行在用户空间，位于 UDP 之上。它的操作方式如下所述。多媒体应用由多个音频、视频、文本或可能其他流组成。这些流被送入到 RTP 库中，RTP 库和多媒体应用一起都位于用户空间。然后，RTP 库将这些流复用并编码到 RTP 数据包中，RTP 数据包再被塞进到一个套接字中。在套接字的操作系统那一端，RTP 数据包被封装到新生

成的 UDP 数据包，这些 UDP 数据包被交给 IP，以便发送到链路上，比如以太网。在接收端的处理与发送端的处理次序相反。最终多媒体应用从 RTP 库函数接收到多媒体数据，它负责播放所收到的媒体。这种情况的协议栈如图 6-30（a）所示。数据包的嵌套封装情况如图 6-30（b）所示。

这种设计的后果是很难说清楚 RTP 位于哪一层。由于它运行在用户空间，并且与应用程序链接，所以它无疑看起来像是一个应用协议。另一方面，它又是一个与具体应用无关的通用协议，它仅提供了一些传输设施，所以它看起来也像一个传输协议。可能最恰当的描述是：它是一个在应用层上实现的传输协议，这就是为什么我们在本章涉及它的原因。

RTP—实时传输协议

RTP 的基本功能是将几个实时数据流复用到一个 UDP 数据包流中。这个 UDP 流可以被发送给一台目标主机（单播传输模式），也可以被发送给多台目标主机（组播传输模式）。因为 RTP 仅仅使用了常规的 UDP，所以路由器不会对它的数据包有任何特殊的对待，除非开通了某些通常的 IP 服务质量特性。特别地，这里没有任何保障传递可靠性的措施，数据包可能会丢失、延迟或者损坏等。

RTP 格式包含了几种有助于接收端处理流媒体信息的特性。在 RTP 流中发送的每个数据包被赋予一个编号，而且每个数据包的编号比它前面的数据包编号大 1。这种编号使得接收方能够确定是否有数据包丢失。如果一个数据包丢失，则接收方能够采取的最佳动作是交给应用程序来处理。如果数据包携带的是视频数据，或许可以跳过该视频帧；如果数据包携带的是音频数据，或许可以利用插值法近似地估计出已丢失的值。重传不是一种切合实际的选择方式，因为重传的数据包可能到达得太晚以至于不再有用。因此，RTP 没有确认，也没有请求重传的机制。

每个 RTP 有效载荷可能包含多个样本，它们可以按照应用系统希望的任何一种方式进行编码。为了允许网络互连，RTP 定义了几种配置轮廓（比如单音频流）；而且对于每一种轮廓，又可以允许多种编码格式。例如，一个单音频流的编码方式包括使用增量编码以 8 kHz 的 8 位 PCM 采样、预测编码、GSM 编码、MP3 编码等。RTP 在头中提供了一个字段让源端用来指定编码方法；除此以外，RTP 不涉及如何进行编码。

许多实时应用需要的另一种设施是时间戳机制。这里的想法是允许源端将每个数据包中时间戳与第一个样本关联起来。这里的时间戳是相对于整个流的起始时间，因此，只有时间戳之间的差值才是重要的，时间戳的绝对值没有任何意义。就像我们马上要描述的那样，这种机制使得接收方可以做少量的缓冲工作，然后在整个流开始之后的正确毫秒时间点上播放每一个样本，并且独立于每个样本所在数据包的到达时间。

时间戳机制不仅减小了网络延迟变化的影响，而且还允许在多个流之间实行同步。例如，一个数字电视节目可能有一个视频流和两个音频流。两个音频流中的内容可能是立体声广播，也可能是电影节目中的两个语言声道（一个声道是原始语种的声音，另一个声道是本地语言的配音），从而让观众有选择语言的机会。每个流分别来自于不同的物理设备；但是，如果它们的时间戳始于同一个计数器，那么，即使这些流的传输和/或者接收过程稍微有一点不规律，它们仍然可以非常同步地播放出来。

RTP 的头结构如图 6-31 所示。它包含 3 个 32 位的字，并有潜在的一些扩展。第一

个字包含了版本字段，现在版本号已经达到了 2。我们希望这个版本非常接近于最终的版本，因为这里只剩下一个值了（虽然可以将 3 定义成说明版本号在一个扩展字中的特殊含义）。



图 6-31 RTP 头格式

P 位表示该数据包被填充到了 4 字节的倍数。最后的填充字节指明了有多少个字节被填充进来。X 位表示有一个扩展头。扩展头的格式和含义没有定义。唯一定义的事情是扩展头的第一个字给出了扩展头的长度。这是针对任何不可预见之需求的最后退路。

CC 字段指明了后面共有多少个贡献源（contributing source），可以从 0 到 15（见下面的说明）。M 位是一个与应用相关的标记位，可以用来标记一个视频帧的开始、音频信道中一个字的开始，或者其他由应用解释的某些事情。有效载荷类型（Payload type）说明使用了哪一种编码算法（比如未压缩的 8 位音频、MP3 等）。由于每个数据包都带有这个字段，所以在传输过程中可以改变编码方法。序号（Sequence number）只是一个计数器，每发送一个 RTP 数据包该计数器都要递增。该字段可被用来检测数据包的丢失。

时间戳（Timestamp）由媒体流的源端产生，它注明了数据包中第一个样本什么时候产生的。这个值有助于减缓接收端的时间变化——称为抖动（jitter），具体做法是将播放与数据包的到达时间分离开。同步源标识符（Synchronization source identifier）指明了该数据包属于哪一个流。正是通过这个字段，可以将多个数据流复用到一个 UDP 数据包流中，或者从一个 UDP 数据包流中分出多个数据流。最后，如果在演播室中使用了混合器，则要使用贡献源标识符（Contributing source identifier）。在这种情况下，混合器是同步源，被混合的流就列在这里（即贡献源标识符）。

RTCP——实时传输控制协议

RTP 有一个姊妹协议，称为实时传输控制协议（RTCP, Realtime Transport Control Protocol）。它和 RTP 一起由 RFC 3550 描述。RTCP 能处理反馈、同步和用户接口等信息，但不传输任何媒体样值。

RTCP 的第一个功能是可以向源端提供有关延迟、抖动、带宽、拥塞和其他网络特性的反馈信息。编码进程充分利用了这些信息，当网络状况比较好的时候，它提高数据速率（从而达到更好的质量）；而当网络状况不好时候，它减低数据速率。根据连续的反馈信息，编码算法可以不断地调整，从而在当前条件下提供尽可能最好的质量。例如，如果在传输过程中带宽增加或者减少，那么编码算法可以根据需要从 MP3 切换到 8 位 PCM，或者切换到增量编码。利用有效载荷类型字段可以告诉接收方当前数据包使用的是哪一种编码算

法，因而有可能根据需要动态地改变编码算法。

提供反馈信息的一个问题是要求所有参与者发送 RTCP 报告。对于一个规模较大的组播应用，RTCP 协议所使用的带宽会变得很大。为了防止这种情况发生，RTCP 发送端缩减其报告率，使得总的带宽不超过媒体带宽的 5%。要做到这一点，每个参与者需要了解媒体的带宽和参与者的数目。对于媒体带宽，它可从发送端那里学习到这点；对于参与人数，它可以通过监听其他参与者的 RTCP 报告估算出来。

RTCP 也处理流之间的同步。问题在于不同的流有可能使用不同的时钟，并且有不同的粒度和不同的漂移速率。利用 RTCP 可使它们保持同步。

最后，RTCP 还提供了一种命名不同源的方法（比如使用 ASCII 字符）。这些信息可以显示在接收端的屏幕上，告诉用户此刻在跟谁通话。

有关 RTCP 的更多信息，请参考（Perkins，2003）。

带有缓冲和抖动控制的播放

一旦媒体信息到达接收端，它必须在合适的时间播放出来。一般情况下，这个时间不是 RTP 包到达接收端的时间，因为数据包通过网络传输所需要的时间略有不同。即使在发送端，数据包以正确的时间间隔被依次注入网络，它们到达接收端的相对时间也不同。这种延迟的变化称为抖动（jitter）。即使是少量的数据包抖动，如果简单地按它到达的时间播放出来，也可能导致媒体成品发散，如抖动的画面帧和难以辨认的音频。

这个问题的解决办法是在接收端播放媒体之前对其进行缓冲，以此来减少抖动。图 6-32 给出了一个实例，我们看到一个数据包流在传输过程中遭受了很大的抖动。在 $t=0$ 时刻，服务器发出数据包 1，该数据包在 $t=1$ 时刻到达客户端；随后的数据包 2 遭受了更多的延迟，它经过 2 秒才到达客户端。随着数据包的到达，它们均被缓冲在客户端机器上。

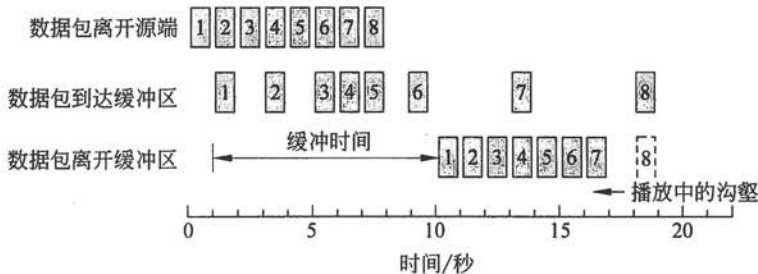


图 6-32 通过缓冲数据包来平滑输出流

在 $t=10$ 秒时，播放器开始播放媒体。这时，从数据包 1 到数据包 6 都已经在缓冲区中，因此它们以平滑播放的统一间隔从缓冲区中移出。一般情况下，没有必要使用统一的间隔，因为 RTP 的时间戳告诉了播放器应该在何时播放相应的媒体。

不幸的是，我们可以看到数据包 8 被延迟得太多，当播放到它时还不可用。这种情况下可以有两种选择。第一种是跳过数据包 8 继续播放后续的数据包。另一种方法是停止播放，直到数据包 8 到达，此时在播放的音乐或电影中就会出现一个恼人的间隙。在现场直播的媒体应用中，比如语音 IP 电话，延迟的数据包通常会被跳过去。直播应用在暂缓播放时无法很好地工作。在流媒体应用中，播放器或许可以暂停。通过延迟媒体的开始播放时间，可以缓解这个问题，但为此需要使用较大的缓冲区。对于流式音频或视频播放器，经

常采用约 10 秒的缓冲区, 以确保播放器按时接收所有数据包 (即那些没有被网络丢弃的数据包)。对于像视频会议这样的实时应用, 响应所需要的缓冲区较短。

平滑播出的一个主要考虑因素是播放点 (playback point) 设置在哪里, 或接收器在播放媒体之前要等待多久。决定等待多长时间要取决于抖动。低抖动和高抖动连接之间的区别如图 6-33 所示。两者之间的平均延迟没有很大的不同, 但如果存在高抖动, 则播放点可能需要比低抖动多捕获超过 99% 的数据包。

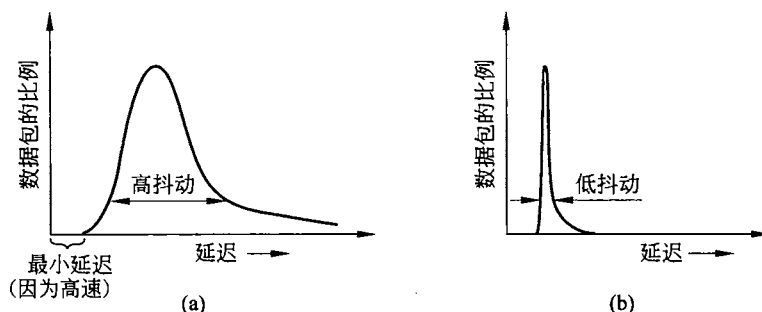


图 6-33
(a) 高抖动; (b) 低抖动

为了寻找好的播放点, 应用程序可以测量抖动, 即考查 RTP 时间戳和数据包到达时间之间的差异。每个差值给出了延迟的一个样本值 (加上一个任意值, 得到固定偏移)。然而, 由于流量竞争和路由变化等其他原因, 延迟可能随时改变。为了适应这种变化, 应用程序在运行时可以自适应播放点。但是, 如果做得不好, 改变播放点可能会产生用户观察到的毛刺。对于音频来说, 避免这个问题的一种方法是适应音频会话峰 (talkspurts) 之间的播放点, 即交谈中的间隙。没有人会注意到一个短的和稍长一点静默之间的差异。RTP 协议允许应用程序为此目的设置 M 标志位, 表明一个音频会话峰的开始。

如果数据包的绝对延迟太长, 以至于媒体播放时还没到来, 那么直播应用将受到影响。如果已经使用了直接路径, 那么无论做什么都无法减少传播延迟。可以简单地拖曳住播放点不动, 然后接受太晚到达的大部分延迟数据包。如果这种处理方式不能接受, 那么拽住播放点的唯一途径是通过使用一种更优质的服务来减少抖动, 例如, 加速转发的区分服务。也就是说, 需要更好的网络。

6.5 Internet 传输协议: TCP

UDP 是一个简单的协议, 它有一些非常重要的用途, 比如客户-服务器交互和多媒体应用, 但是对于大多数 Internet 应用来说, 它们需要可靠的、按序递交的传输特性。UDP 不能提供这样的功能, 所以 Internet 还需要另一个协议。这就是 TCP; 它是 Internet 上的主力军。现在让我们来详细学习这个协议。

6.5.1 TCP 概述

传输控制协议 (TCP, Transmission Control Protocol) 是为了在不可靠的互联网络上提

供可靠的端到端字节流而专门设计的一个传输协议。互联网络与单个网络有很大的不同，因为互联网络的不同部分可能有截然不同的拓扑结构、带宽、延迟、数据包大小和其他参数。TCP 的设计目标是能够动态地适应互联网络的这些特性，而且具备面对各种故障时的健壮性。

TCP 的正式定义由 1981 年 9 月的 RFC 793 给出。随着时间的推移，已经对其做了许多改进，各种错误和不一致的地方逐渐被修复。为了让你感受到 TCP 的扩展历程，现在重要的 RFC 有：RFC 793 plus：澄清了说明，RFC 1122 修复了 bug、RFC 1323 做了高性能扩展，RFC 2018 定义了选择性确认，RFC 2581 说明了拥塞控制、RFC 2873 定义了为服务质量而重用的头字段，RFC 2988 改进了重传计时器，RFC 3168 定义了显式拥塞通知。完整的协议集合很大，因而专门发布了一个针对许多 RFC 的指南，它就是作为另一个 RFC 文档公布的 RFC 4614。

每台支持 TCP 的机器都有一个 TCP 传输实体。TCP 实体可以是一个库过程、一个用户进程，或者内核的一部分。在所有这些情形下，它管理 TCP 流，以及与 IP 层之间的接口。TCP 传输实体接受本地进程的用户数据流，将它们分割成不超过 64 KB（实际上去掉 IP 和 TCP 头，通常不超过 1460 数据字节）的分段，每个分段以单独的 IP 数据报形式发送。当包含 TCP 数据的数据报到达一台机器时，它们被递交给 TCP 传输实体，TCP 传输实体重构出原始的字节流。为简化起见，我们有时候仅仅用“TCP”来代表 TCP 传输实体（一段软件）或者 TCP 协议（一组规则）。根据上下文语义你应该能很清楚地推断出其实际含义。例如，在“用户将数据交给 TCP”这句话中，很显然这里指的是 TCP 传输实体。

IP 层并不保证数据报一定被正确地递交到接收方，也不指示数据报的发送速度有多快。正是 TCP 负责既要足够快地发送数据报，以便使用网络容量，但又不能引起网络拥塞；而且，TCP 超时后，要重传没有递交的数据报。即使被正确递交的数据报，也可能存在错序的问题，这也是 TCP 的责任，它必须把接收到的数据报重新装配成正确的顺序。简而言之，TCP 必须提供可靠性的良好性能，这正是大多数用户所期望的而 IP 又没有提供的功能。

6.5.2 TCP 服务模型

TCP 服务由发送端和接收端创建一种称为套接字（socket）的端点来获得，正如我们在 6.1.3 节中所讨论的那样。每个套接字有一个套接字编号（地址），该编号由主机的 IP 地址以及一个本地主机的 16 位数值组成的。这个 16 位数值称为端口（port），端口是 TCP 的 TSAP 名字。为了获得 TCP 服务，必须显式地在一台机器的套接字和另一台机器的套接字之间建立一个连接，有关套接字的调用如图 6-5 中所列。

一个套接字有可能同时被用于多个连接。换句话说，两个或者多个连接可能终止于同一个套接字。每个连接可以用两端的套接字标识符来标识，即（socket1, socket2）。TCP 不使用虚电路号或者其他的标识符。

1024 以下的端口号被保留，只能用作由特权用户（比如 UNIX 系统的 root）启动的标准服务。这些端口称为知名端口（well-known port）。例如，如果一台机器上的任何一个进程希望检索邮件，那么它可以连接到目标主机的 143 号端口与 IMAP 守护进程联系。可以在 www.iana.org 上找到所有知名端口的列表。目前已经分配了 700 多个。图 6-34 列出了一